



Research paper

IK-Geo: Unified robot inverse kinematics using subproblem decomposition

Alexander J. Elias^{ID}*, John T. Wen^{ID}

Department of Electrical, Computer, and Systems Engineering, Rensselaer Polytechnic Institute, 110 8th St, Troy, NY 12180, United States



ARTICLE INFO

Keywords:

Computational geometry
Industrial robots
Kinematics
Offset wrist

ABSTRACT

We present IK-Geo, a highly capable and computationally efficient open-source robot inverse kinematics (IK) solver. In this unifying approach, IK for any 6-DOF all-revolute (6R) manipulator is decomposed into six canonical geometric subproblems solved by intersecting circles with other geometric objects. Subproblems are efficiently solved in all cases including in a continuous and sometimes least-squares sense when a solution does not exist. This continuity requirement means IK-Geo finds all IK solutions including singular solutions and sometimes least-squares solutions. Robots with three intersecting or parallel axes are solved in closed form. All other commercially available robots have at least one pair of intersecting or parallel axes and are solved by searching over one joint angle. Fully general robots are solved by searching over two joint angles. Search solutions may be converted to a system of three or four polynomials in terms of the end effector pose in the tangent half-angle of one joint. A comparison with IKFast and the MATLAB Robotics Toolbox IK solver demonstrates that IK-Geo has faster computation and can solve more classes of robots.

1. Introduction

Robotics applications have always demanded a fast and reliable inverse kinematics (IK) algorithm that ideally returns all solutions for a given end effector pose. Fast IK is crucial for real-time Cartesian control and path planning as well as for offline conversion of trajectories from task space to joint space. The ability to return all IK solutions is critical for evaluating all possible robot configurations during path planning, and singularity robustness helps avoid resorting to joint-space control to switch between IK solution branches.

There are many classes of robot kinematics based on intersecting or parallel axes as shown in Table 1. Nonconsecutive intersecting joint axes $k, k+2$ occur when they are both orthogonal to axis $k+1$ or when the two links have equal lengths, zero offset, and opposite twists, which is a special case of the Bennett criteria [1]. An important requirement for an IK solver is not only to solve the general 6-DOF all-revolute (6R) case but also to use simplifications arising from intersecting or parallel axes to improve computational performance.

Gradient-based solvers, such as the popular solver TRAC-IK [11], use the robot Jacobian and can run sufficiently quickly for real-time control. However, they only return one solution close to the initial guess, suffer from performance issues close to robot singularities, and may not always return solutions within a given time period, especially if there is no close initial guess or if there is a high accuracy requirement.

In contrast, analytical IK solvers can return all inverse kinematics solutions and can run much faster than gradient-based solvers. IKFast [12] is the most widely used analytical IK solver. However, IKFast cannot solve all 6R robots, and it depends on

* Corresponding author.

E-mail addresses: eliasa3@rpi.edu (A.J. Elias), wenj@rpi.edu (J.T. Wen).

Table 1

Classification of all 6R robot manipulator kinematics. Some examples are 7R robots with one fixed joint angle where the kinematic family may change depending on which joint angle is fixed.

Robot kinematic family	Example
Spherical joint and two intersecting axes and two parallel axes	Franka Production 3 [2], fixed q_5 KUKA LBR iiwa 7 R800 [3], fixed q_3 ABB IRB 6640 [4]
Three parallel axes and two intersecting axes and two parallel axes	N/A ^a Universal Robots UR5 [5] N/A ^a
Two intersecting axes and two intersecting axes and two parallel axes	Kassow Robots KR810 [6], fixed q_7 FANUC CRX-10iA/L [7] Kawasaki KJ125 [8]
Two parallel axes and two parallel axes	N/A ^a N/A ^a
Two intersecting axes $k, k+2^b$ and two intersecting axes and two parallel axes	ABB YuMi [9], fixed q_3 RRC K-1207i [10], fixed q_6 N/A ^a
General 6R	Kassow Robots KR810 [6], fixed q_6

^a No such industrial robot.

^b Axes k and $k+2$ are nonconsecutive intersecting axes. All other intersecting or parallel cases refer to consecutive axes.

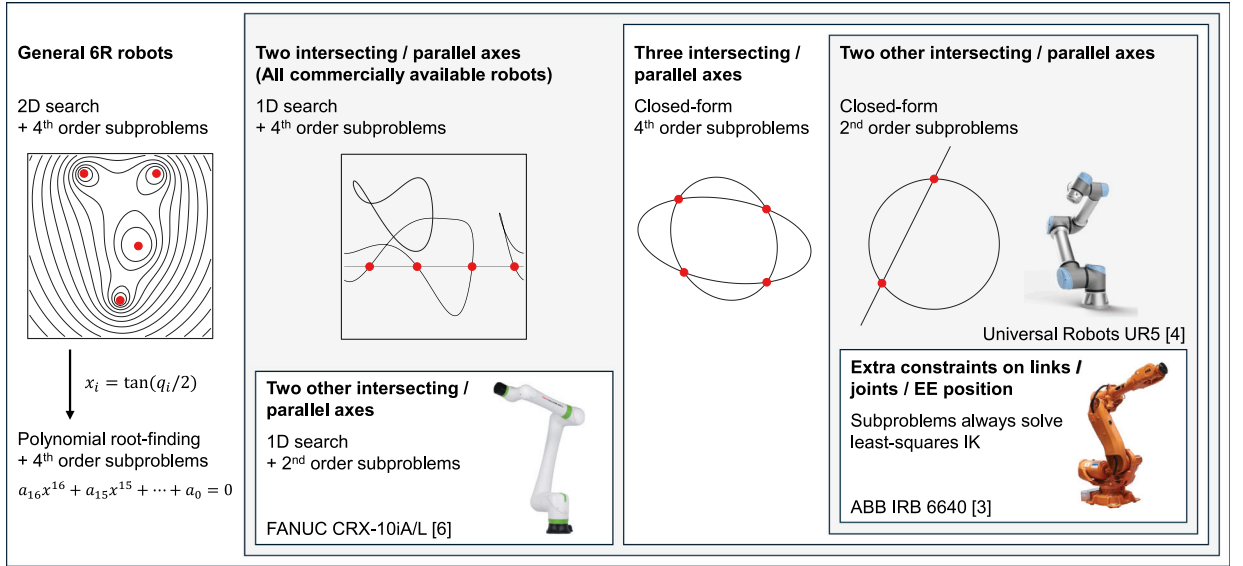


Fig. 1. IK-Geo is an analytical and semi-analytical solver that exploits intersecting and parallel joint axes to optimize the solution method.

symbolic manipulation techniques and exploring possible simplifications, meaning code generation and even compilation can take a considerable amount of time.

Other IK methods exist but also have limitations. AI algorithms [13,14] require extensive tuning and training and do not return precise results. General 6R methods based on tangent half-angle polynomials are difficult to implement in real-time settings because they require performing symbolic algebraic manipulations for each new end effector pose.

In this paper, we present IK-Geo, a highly capable analytical and semi-analytical general IK solver for 6R arms based on a unifying and simplifying geometric approach called subproblem decomposition (Fig. 1). This method solves for a small number of joint angles at a time by exploiting simplifications arising from intersecting or parallel joint axes and rewriting the problem as a series of a few canonical geometric subproblems with known solutions, shown in Table 2. (Details of the subproblem formulations are discussed in Section 2.) This results in algorithms that are precise, computationally efficient, stable, and that return all solutions, including singular solutions and sometimes least squares solutions. The IK algorithms find a continuous joint trajectory for any given continuous end-effector trajectory even if it falls outside the robot workspace because the subproblems return approximate but continuous solutions when a solution does not exist. In cases of an internal robot singularity where a continuum of self-motion is possible, such as when joint axes are collinear, a subproblem may have a continuum of solutions rather than finitely many solutions. To achieve continuous joint trajectories, redundancy is resolved by picking from the continuum of subproblem solutions.

Table 2
Subproblem formulations.

Subproblem	Equation
1: Circle and point	$\min \ R(k, \theta)p_1 - p_2\ $
2: Two circles	$\min \ R(k_1, \theta_1)p_1 - R(k_2, \theta_2)p_2\ $
3: Circle and sphere	$\min \ R(k, \theta)p_1 - p_2\ - d$
4: Circle and plane	$\min h^T R(k, \theta)p - d $
5: Three circles ^a	$p_0 + R(k_1, \theta_1)p_1 = R(k_2, \theta_2)(p_2 + R(k_3, \theta_3)p_3)$
6: Four circles ^a	$\begin{cases} h_1^T R(k_1, \theta_1)p_1 + h_2^T R(k_2, \theta_2)p_2 = d_1 \\ h_3^T R(k_3, \theta_1)p_3 + h_4^T R(k_4, \theta_2)p_4 = d_2 \end{cases}$

^a Returns continuous approximate solutions for singularity robustness.

Table 3
IK solution types and subproblems used.

Solution type	Robot kinematic family	SPs used	Section
Closed-form	Spherical joint	1, 2, 5 ^a	4.1
	Three parallel axes	1, 3, 6 ^a	4.2
1D search	Two intersecting axes	1, 5 ^a	5.1
	Two parallel axes	1, 6 ^a	5.2
	Two intersecting axes $k, k+2$	1, 5 ^a	5.3
2D search	General 6R	1, 5	5.4

^a Simplifies to Subproblems 1–4 if extra intersecting or parallel axes.

IK-Geo solves the IK problem faster than commonly used solvers. For example, based on the performance evaluation presented in Section 6, IK for a UR5 robot using IK-Geo is more than 40 times faster than IKFast. The timing performance was independently verified in [15], where the authors compared closed-form IK-Geo solutions to IKFast as well as Gauss–Newton (Jacobian pseudoinverse) and Levenberg–Marquardt (damped least-squares) iterative solutions. Furthermore, [15] shows IK-Geo has superior solution accuracy compared to the other algorithms. Depending on cases of intersecting or parallel joint axes, robot IK is solved in closed form or with a reduced dimension search over one or two joint angles, as shown in Table 3. To the best of our knowledge, all commercially available industrial 6R robots and 7R robots parameterized by some joint angle have intersecting or parallel axes and therefore can be solved in closed form or with 1D search. The closed-form method applies to any 6R robot with three consecutive intersecting or parallel axes, commonly referred to as the Pieper criterion [16], and requires only addition, subtraction, multiplication, division, square roots, ATAN2, and (in the case of solving quartic polynomials) cube roots. Otherwise, searching over just one or two joint angles is much more efficient than searching over all six joint angles as in Jacobian-based methods. The 1D and 2D search cases also offer a useful graphical output. To the best of our knowledge, we are the first paper to apply only subproblems to solve IK for any general 6R robot with 2D search, any 6R robot with any two intersecting or parallel axes with 1D search, including when the intersecting axes are nonconsecutive, and any 6R robot with three parallel axes in closed form.

Beyond its performance benefits, IK-Geo is open-source and convenient to use. Implementing the subproblem decomposition algorithms takes few lines of code, so the code is easy to understand, modify, port, and debug. The IK solutions are not based on symbolic simplification exploration, so there is no setup or code generation step. In fact, the same code can apply to any robot in the same kinematic family, even if the exact link lengths or joint offsets are different.

We are also the first to make a connection between subproblem decomposition and tangent half-angle polynomial IK methods which guarantee finding all solutions. Unlike other polynomial methods, we arrive at the univariate polynomial in few steps and directly use simplifications arising from intersecting and parallel axes. Using the same subproblem decompositions and error function formulations as in the search method, the tangent half-angle substitution results in a system of three or four multivariate polynomials. Eliminating variables leads to the resultant univariate polynomial which may be solved to arbitrary precision. Once one or two joint angles are found, the remaining joint angles are found in closed form. Although the polynomial method is less efficient than search methods, we share insights on improving computational performance by efficiently parameterizing end effector orientation and by using rational approximations of kinematic parameters and end effector pose.

The subproblem decomposition method is not limited to 6R IK. The subproblem decomposition method has also been used for IK of parallel robots and robots with prismatic joints [17]. In [18], we show the subproblem decomposition methods presented in this paper, including the search and polynomial methods, can be used for IK for 7R manipulators parameterized by shoulder-elbow-wrist (SEW) angle, also known as elbow angle or swivel angle. Although we do not provide details in this paper, the subproblem decomposition method may also be used to solve the forward kinematics of parallel manipulators as well as other geometric problems such as in computer graphics.

The remaining sections of this paper are as follows. We provide an overview of the history of inverse kinematics and the subproblem decomposition method in Section 2 and then explain our IK approach in Section 3. In Sections 4 and 5, we provide closed-form and search-based IK solutions, and in Section 6 we demonstrate significant performance speedups compared to alternatives like IKFast. In Section 7 we explain our method of IK by finding zeros of the polynomial in the tangent half-angle.

We conclude and discuss ideas for improving the algorithms even further in Section 8. The six subproblems are solved in [Appendix A](#), and numerical examples for the polynomial approach are shown in [Appendix B](#).

The IK solutions, subproblem solutions, and timing tests are coded in MATLAB and Rust. The polynomial method is implemented in Mathematica and Maple. All code is open-source and available on a publicly accessible repository [19].

2. Related work

This work is a continuation of a long line of work in analytical and semi-analytical (i.e., reduced-dimension search) solutions to the inverse kinematics problem. The seminal thesis by Pieper [16] presented analytical solutions for many special robot cases, and for the general 6R robot case, he showed the polynomial in the tangent half-angle is at most degree 524,288. Since then, work on the IK problem by other authors has tended to focus on two areas: The general 6R robot IK problem and simplified kinematics IK problems.

To solve the general 6R robot IK problem, authors commonly find a 16th-order polynomial whose real roots correspond to the (up to) 16 solutions for such an arm. Although the general 6-DOF IK problem has been solved, solutions often require computer-assisted algebra or the tangent half-angle substitution, so there is still a need for more robust, faster, and more easily understood algorithms. Much of this work was published in the early 1990s. [20,21] were one of the first to devise a procedure to construct a 16th-order polynomial in the tangent half-angle of one of the joints. Many others quickly followed with simpler and more efficient procedures. [22] found a 16th-order polynomial differently, [23] reduced general 6R IK to an eigenvalue problem or generalized eigenvalue problem, and [24] reduced the problem to a 12×12 quadratic eigenvalue problem. More recently in 2007, [25] solved the general 6R IK problem using dual quaternions and 7D projective geometry. They reduced the IK problem to the intersection of eight hyperplanes and a six-dimensional quadric which is solved by finding a 48-degree polynomial and factoring out a 16th-order polynomial. Even today, methods based on tangent half-angle polynomials are difficult to implement in real-time settings because they require performing symbolic algebraic manipulations for each new end effector pose. A 2D search-based method is discussed in [26] (they call it the bivariate-equation approach) where two joint angles are searched over, and the tangent half-angle substitution is not used. In [27], the 2D search method presented by [26] was extended by efficiently finding all solutions using Bernstein elimination, but symbolic algebra must still be performed for each new end effector pose.

For certain robot geometries, there are fewer than 16 solutions, and the order of the polynomial in the tangent half-angle decreases [28]. In some cases, the solution may be simplified down to solving quartic or even quadratic polynomials. While using the general 6R methods for simplified robot cases is viable, the solutions tend to be much easier when using methods specific to those cases. Pieper [16] focused much of his work on the cases of three intersecting axes because the rotation and position parts of the IK problem become decoupled: The three joint angles not part of the spherical joint are determined first, and then the three spherical joint angles are determined as in the solution of Euler angles. The three parallel axes case is similar, as the three axes still intersect but at a point at infinity. Search may also be used to solve such simpler robots, and in [29,30], 1D search is used to solve IK for some simple manipulators with many intersecting or parallel axes.

IKFast [12], the most widely used analytical IK solver, is part of the OpenRave project and has been used in other large projects such as MoveIt [31] and Tesseract [32]. The original implementation of IKFast could find IK solutions for robots with three intersecting axes by decomposing the kinematics into a position and rotation part and solving them separately using symbolic manipulation. More recent updates to the program code also include implementations of the general 6-DOF IK algorithms presented in [22–24]. IKFast is a Python file nearly 10 000 lines long which generates an even larger C++ file, ranging from about 10 000 lines to over 100 000 lines in the case of a UR5 robot. This means modifying, optimizing, or debugging by hand is often impractical. Code must be regenerated for different kinematics parameters even if only link lengths differ, and solutions break down at singularities. IKBT [33] is a newer analytical IK solver similar to IKFast, but it is still based on symbolic manipulation and simplification exploration, so it suffers from similar issues.

Paden [34] and Kahan [35] showed that for some 6R robots, the inverse kinematics problem may be decomposed into a series of smaller canonical subproblems with closed-form solutions. The influential robotics textbook [17] called these subproblems the Paden–Kahan Subproblems 1, 2, and 3. However, Subproblems 1–3 alone cannot solve all 6R robots with three parallel or intersecting axes.

To solve the IK problem for any 6R arm, we identify and solve six canonical geometric subproblems, shown in [Table 2](#). Subproblems 1–3 are the least-squares versions of the original Paden–Kahan subproblems, and Subproblem 4 is used when there are two parallel axes in a robot or in solving a spherical joint. Subproblems 5 and 6 are used in the case of three intersecting or parallel axes, respectively. Subproblem 5 has been identified by many names including the position problem [26], the third order subproblem [36], the three-joint subproblem [37], and the wrist center positioning problem [38].

We are careful to pose the subproblems so that they are robust to singularities through a continuity requirement, and to our knowledge, we are the first to solve these subproblems in this way. We pose the subproblems to return an approximate solution for any branch without an exact solution, and the mapping between the subproblem parameters and the returned solutions must be continuous. For Subproblems 1–4, we make this requirement stronger and require least-squares solutions. Subproblems must also identify and solve cases where there is a continuum of solutions.

In the subproblem formulations, θ is an unknown angle, p is an $\mathbb{R}^3$ vector, h is a unit $\mathbb{R}^3$ vector, and d is a scalar. (d must be nonnegative in Subproblem 3.) Note that $R(k, \theta)$ denotes the rotation matrix about a unit vector $k \in \mathbb{R}^3$ over an angle θ . For some fixed p, k and parameter θ , $R(k, \theta)p$ sweeps out the circle on the edge of a right circular cone with axis k and generator p . We also consider generalizations of Subproblem 2 where the circle axes do not intersect. As long as there is no continuum of solutions,

Subproblem 1 has one exact solution or one least-squares solution, Subproblems 2, 3, and 4 have up to two exact solutions or one least-squares solution (Subproblem 2 has up to two least-squares solutions if $\|p_1\| \neq \|p_2\|$), and Subproblems 5 and 6 have up to four solutions.

Various authors have contributed solutions to the original three Paden–Kahan Subproblems as well as Subproblems 4–6, either by explicitly identifying them as subproblem solutions or by addressing them while solving IK problems. (Authors tend to solve the exact versions rather than the least-squares or continuous versions of these subproblems.) In [36,37,39–53], subproblems are solved using screw theory. In [50], subproblem solutions are provided with quaternions, and in [50,54–56] dual quaternions are used. In [40,42,43,49,53], a new subproblem of two consecutive rotations with an offset is posed, which is a generalization Subproblem 2. In [44,47,53,56], the solution for two consecutive parallel rotations is provided, which is a special case of this generalization. Subproblem 4 has been solved before as part of the solution to the inverse kinematics of a spherical wrist, such as in [57]. Several authors posed new subproblems with three consecutive rotations with constraints on which axes are parallel or intersecting, which are special cases of Subproblem 5 [39,41,44,45,47,49,53,55]. In [26,36–38], the fully general case of three consecutive rotations is solved. In [58], a specific instance of Subproblem 5 was solved for IK of a 7R arm, and the authors noticed that the problem reduces to the intersection of two ellipses. The general 6-DOF methods presented in [25] simplify when a robot has three intersecting or parallel axes, meaning this method can solve Subproblems 5 and 6, as elaborated on in [59]. In solving Subproblem 5, the simplifying cases of two parallel or intersecting axes means authors also solve Subproblems 2 and 4. In [26], the author reduces Subproblem 5 to Subproblem 6, and so inadvertently solves Subproblem 6 as well.

The algorithms presented in Appendix A solve the least-squares or continuous approximate versions of the subproblems and identify when these conditions occur. The algorithms also identify conditions for a continuity of solutions. This leads to robustness to singularities in the IK algorithms. These algorithms do not resort to the tangent half-angle substitution, instead relying on ATAN2 as the only inverse trigonometric function, leading to high precision. The algorithms result in extremely efficient code both in terms of computational efficiency as well as length of code.

To demonstrate the computational performance and range of robot kinematics that the subproblem decomposition method offers, we will compare performance to IKFast and to a MATLAB implementation of Pieper’s method. MATLAB’s Robotics System Toolbox [60] closed-form inverse kinematics solver, `analyticalInverseKinematics`, is based on the Pieper method [16] and is designed to work with any robot with a spherical wrist. The solver generates a MATLAB function that finds all inverse kinematics solutions and can then be converted to a C++ MEX function with MATLAB Coder.

3. Subproblem decomposition approach to inverse kinematics

Consider a 6-DOF articulated robot with all revolute joints as shown in Fig. 2. We will use the product of exponentials convention to describe the arm kinematics: The kinematics is described using a series of coordinate frames each defined by a set of three orthonormal vectors (e_x , e_y , e_z). Each coordinate frame corresponds to a frame origin $\mathcal{O}$. Let frame 0 be the world coordinate frame, let frame i , $i = 1, \dots, 6$ be the coordinate frame representing the orientation of link i , and let frame T be the coordinate frame representing the orientation of the end effector tool. The zero configuration where all joint angles are equal to zero can be defined arbitrarily, and in this configuration all link frames have orientation equal to the world frame. (In the product of exponentials convention, rotations do not necessarily occur about the z axis of the previous frame.) Let $\mathcal{O}_0$ be the origin of the world frame, let $\mathcal{O}_i$, $i = 1, \dots, 6$, be the origin of frame i , placed anywhere along the i th joint axis, and let $\mathcal{O}_T$ be the origin of the end effector tool frame. Define $p_{i,j}$ ($p_{i,T}$), $i, j = 0, \dots, 6$, be the $\mathbb{R}^3$ vector from $\mathcal{O}_i$ to $\mathcal{O}_j$ ($\mathcal{O}_T$) represented in the i frame. This means $p_{i-1,i}$, $i = 1, \dots, 6$, and $p_{6,T}$ are constant $\mathbb{R}^3$ vectors.

Define h_i to be the constant unit vector rotation axis for the i th joint represented in the i frame, and let $R_{i-1,i} = R(h_i, q_i)$, where q_i is the i th joint angle. Also let $R_{i,i-1} = R(h_i, -q_i)$, and for $i < j$, $R_{i,j} = R_{i,i+1} \cdots R_{j-1,j}$. Finally, define R_{6T} to be the constant orientation of the tool frame with respect to the 6 frame. Then, the rotation and position of the tool frame with respect to the world frame are

$$R_{0T} = R_{01} R_{12} R_{23} R_{34} R_{45} R_{56} R_{6T}, \quad (1a)$$

$$p_{0T} = p_{01} + R_{01} p_{12} + R_{02} p_{23} + R_{03} p_{34} + R_{04} p_{45} + R_{05} p_{56} + R_{06} p_{6T}. \quad (1b)$$

The inverse kinematics problem for a robot with kinematic parameters $(\{p_{i-1,i}\}_{i=1}^6, p_{6T}, \{h_i\}_{i=0}^6, R_{6T})$ means finding all joint angles $q = [q_1 \ q_2 \ \cdots \ q_6]^T$ corresponding to a given end effector rotation and position (R_{0T}, p_{0T}) . In general, one (R_{0T}, p_{0T}) corresponds to multiple q , which are called the different inverse kinematics branches.

Without loss of generality, assume $p_{01} = 0$. (Otherwise, subtract p_{01} from p_{0T} .) Rewriting (1) by moving constants to the left-hand side results in

$$R_{06} = R_{0T} R_{6T}^T = R_{01} R_{12} R_{23} R_{34} R_{45} R_{56}, \quad (2a)$$

$$p_{06} = p_{0T} - R_{06} p_{6T} = R_{01} p_{12} + R_{02} p_{23} + R_{03} p_{34} + R_{04} p_{45} + R_{05} p_{56}. \quad (2b)$$

This means the IK procedures can be in terms of (R_{06}, p_{06}) .

In the next sections we provide inverse kinematics solutions using the subproblem decomposition method. For each robot example, cases of intersecting or parallel axes result in simplifications to (2). For example, if axes $i-1$ and i intersect then choosing $\mathcal{O}_{i-1} = \mathcal{O}_i$ results in $p_{i-1,i} = 0$. Then, we decompose the equations into subproblems by moving terms and multiplying

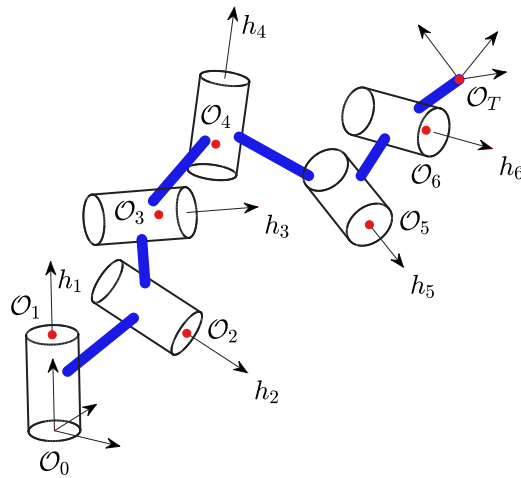


Fig. 2. General 6R robot arm with h_i as the i th joint axes and O_i as the origin of frame i , placed anywhere along h_i . Inverse kinematics can be solved using Subproblem 5 and a 2D search over two joint angles.

by rotations, taking the norm of both sides of the equation, or, in the case of parallel axes, projecting both sides of the equation onto a vector by left multiplying by h_i^T or right multiplying by h_i . This corresponds to solving the equation $R(k, \theta)p_1 = p_2$ (where p_1 and p_2 may be functions of other unknowns) by solving the necessary condition $\|p_1\| = \|p_2\|$ or $k^T p_1 = k^T p_2$. In general, these conditions are not sufficient, so solutions must be checked if they are extraneous. However, any extraneous solutions earlier in the procedure become apparent in the next subproblems, as they do not return any exact solutions.

In some cases, the entire IK problem can be decomposed into only subproblems, which leads to closed-form solutions. In other cases, the decomposition remains in terms of one or two joint angles, so a search method can be used. We also show in Section 7 that rather than search, we can keep the same decomposition and error function but use the tangent half-angle substitution to convert the problem to a system of polynomials.

When a subproblem has multiple solutions, the solution for q branches, meaning the procedures find all inverse kinematics solutions. (If desired, just one of the solutions can be used to speed up computation time.) If a subproblem has a continuum of solutions, such as when two rotation axes are collinear, then the robot has an internal singularity for that branch. This method is robust to such singularities as the arbitrary joint angles can be detected and resolved before continuing the procedure. If a subproblem has no exact solutions, then that branch fails to find a solution; however, using the least-squares solution for Subproblems 1–4 and the similar continuous approximate solutions for Subproblems 5 and 6 are still useful as described in Section 4.3.

The following solutions are valid not just for a single robot, but for a family of robots with certain conditions on their kinematics parameters based on consecutive intersecting or parallel joint axes. Therefore, a single code implementation can perform inverse kinematics for multiple robots in the same kinematic family but with different dimensions.

In the solutions below we demonstrate specific locations where interesting or parallel axes occur. However, similar equations can be easily derived for intersecting or parallel axes occurring elsewhere in the chain.

7R robots have a redundant degree of freedom which may be parameterized by fixing one joint angle. This converts it into a 6R robot which can be solved with the IK solutions below. The kinematic family changes depending on which 7R joint is fixed, and in practice, it is a good choice to fix the joint angle which results in the highest number of intersecting or parallel joint axes. For IK solutions of 7R robots parameterized by shoulder-elbow-wrist (SEW) angle, also known as arm angle or swivel angle, the reader is referred to [18].

4. Closed-form inverse kinematics

If a 6R robot has three consecutive intersecting or parallel joint axes, then the inverse kinematics can be solved using Subproblems 5 or 6, respectively. If a robot has certain other pairs of intersecting or parallel axes, then the solution can be written in terms of just Subproblems 1–4. This is in accordance with the special simplified cases of Subproblems 5 and 6.

4.1. Three intersecting joint axes

When a 6R robot has three intersecting joint axes, the inverse kinematics problem can be decoupled into a position part, which does not depend on the spherical joint, and a rotation part. The position part can be solved with Subproblem 5, and the rotation part can be solved with Subproblems 1 and 2. Consider the common case of a spherical wrist where the last three joint axes (h_4, h_5, h_6)

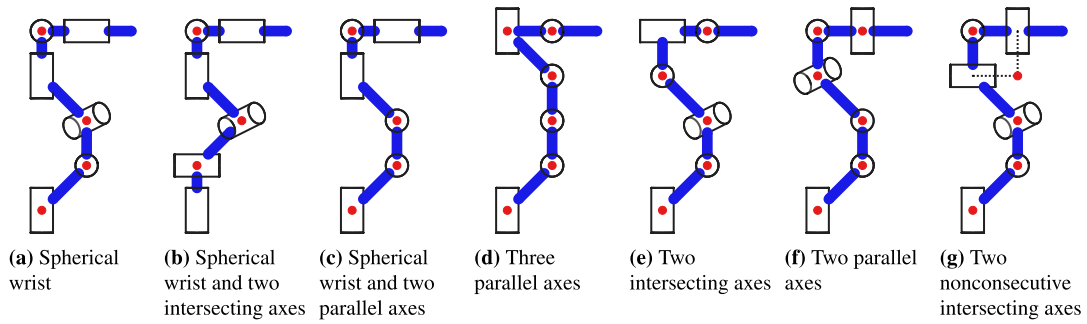


Fig. 3. Some cases for simplified robot inverse kinematics.

intersect at a point, shown in Fig. 3(a). When the spherical joint is at the wrist, the position part depends only on p_{06} and not R_{06} . Choose the origins of frames 4, 5, 6 at the intersection so $p_{45} = p_{56} = 0$. Then, (2b) becomes

$$-p_{12} + R_{10}p_{06} = R_{12}(p_{23} + R_{23}p_{34}), \quad (3)$$

which only depends on (q_1, q_2, q_3) and can be solved using Subproblem 5.

To solve for the wrist angles (q_4, q_5, q_6) , write (2a) as

$$R_{32}R_{21}R_{10}R_{06} = R_{34}R_{45}R_{56}. \quad (4)$$

For each (q_1, q_2, q_3) , the left-hand side is known, which can be called R_{36} . Project it to h_6 :

$$R_{43}R_{36}h_6 = R_{45}h_6, \quad (5)$$

and solve for up to two solutions of (q_4, q_5) using Subproblem 2. Then, find q_6 by using Subproblem 1 to solve

$$R_{56}p = R_{54}R_{43}R_{36}^T p, \quad (6)$$

where p is any vector not collinear with h_6 . For a spherical wrist to achieve any orientation, it must have orthogonal consecutive joint axes.

If a robot has two intersecting axes $(i-1, i)$ separate from the three intersecting axes, then $p_{i-1,i} = 0$ and Subproblem 5 simplifies into Subproblems 2 and 3. Geometrically, this simplification occurs because rotating a point about two intersecting axes keeps the point on a sphere centered at the intersection. Consider a robot with a spherical wrist and the first two axes intersecting as in Fig. 3(b). In this case, $p_{12} = 0$. Taking the norm of both sides of (3), we have

$$\|p_{06}\| = \|p_{23} + R_{23}p_{34}\|, \quad (7)$$

which may be solved using Subproblem 3 for up to two solutions of q_3 . For each q_3 , we can use Subproblem 2 to find up to two solutions of (q_1, q_2) .

If a robot has two parallel axes separate from the three intersecting axes, then Subproblem 5 simplifies into Subproblems 1, 3, and 4. Algebraically, the simplification to (2b) occurs because multiple terms combine once the equation is projected onto the direction of the parallel joint axes. Geometrically, this simplification occurs because rotating a point about two parallel axes keeps the point on a plane normal to the rotation axes. Consider a robot with a spherical wrist and the second and third axes parallel ($h_2 = h_3$) shown in Fig. 3(c). For parallel axes, the choice of link frame origin is arbitrary along the joint axis. Rearranging (3) and projecting onto h_2 , we have

$$h_2^T R_{10}p_{06} = h_2^T (p_{12} + p_{23} + p_{34}), \quad (8)$$

which may be solved using Subproblem 4 for up to two solutions of q_1 . For each q_1 , we can use Subproblem 3 to find up to two solutions of q_3 . For each of the four possible (q_1, q_3) , we can solve for the corresponding q_2 using Subproblem 1.

4.2. Three parallel joint axes

When a 6R robot has three parallel axes, the inverse kinematics problem can be solved using Subproblems 1, 3, and 6. (Such a robot is the limit of a robot with three intersecting axes where the intersection point moves infinitely far away.) Consider a robot with $h_2 = h_3 = h_4$ as shown in Fig. 3(d). Projecting (2b) onto h_2 results in

$$h_2^T R_{10}p_{06} - h_2^T R_{45}p_{56} = h_2^T (p_{12} + p_{23} + p_{34} + p_{45}). \quad (9)$$

From (2a), we have

$$h_2^T R_{10}R_{06}h_6 - h_2^T R_{45}h_6 = 0. \quad (10)$$

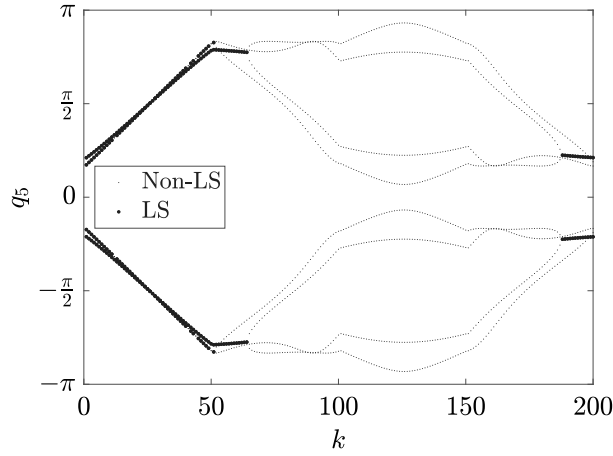


Fig. 4. All IK solutions for the ABB IRB 6640 example path. Only joint angle 5 is plotted among the 6 joint angles in the solution vector. In general, there are 8 IK solutions corresponding to the shoulder being forward or back, the elbow being up or down, and the wrist being flipped or not flipped. At boundary singularities such as the elbow singularity encountered near $k = 0$ to $k = 50$, multiple branches converge to one. Least-squares IK solutions provide numerical stability for poses at the workspace boundary and provide solutions even when they do not exist for poses outside the workspace boundary.

If $p_{56} = 0$, we can find up to two solutions of q_1 from (9) using Subproblem 4. (A similar simplification occurs if a robot has three parallel axes and two other parallel axes.) For each solution of q_1 , we can use Subproblem 4 to solve for up to two solutions of q_5 from (10) for a total of four solutions. If $p_{56} \neq 0$, (9) and (10) form a system of equations that can be solved using Subproblem 6 for up to four solutions of (q_1, q_5) .

By projecting (2a) onto h_6 or h_2 , we can solve for $q_2 + q_3 + q_4$ (and hence R_{14}) and q_6 using Subproblem 1 to solve

$$R_{14}R_{45}h_6 = R_{10}R_{06}h_6, \quad (11)$$

$$R_{65}R_{54}h_2 = R_{06}^T R_{01}h_2. \quad (12)$$

Find up to two solutions of q_3 using Subproblem 3 to solve

$$\|p_{23} + R_{23}p_{34}\| = \|R_{10}p_{06} - p_{12} - R_{14}p_{45} - R_{15}p_{56}\| \quad (13)$$

for a total of eight solutions. Within these eight solutions, we can solve for q_2 using Subproblem 1:

$$R_{12}(p_{23} + R_{23}p_{34}) = R_{10}p_{06} - p_{12} - R_{14}p_{45} - R_{15}p_{56}, \quad (14)$$

and we can solve for q_4 using subtraction, wrapping the result to $[-\pi, \pi]$ if desired.

4.3. Continuous and least-squares inverse kinematics solutions

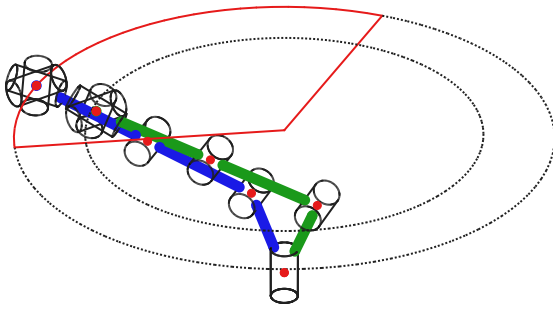
Subproblems 1–4 return least-squares solutions, and Subproblems 5 and 6 also have solution continuity by returning the real part of any complex solutions. This means the IK procedure returns solutions for q even for branches that do not have an exact solution. If a desired end effector pose is not feasible but near the robot workspace, the IK procedure will return a close solution. The desired pose can be exactly on the boundary of the workspace (i.e. at a singularity), and the IK procedure will correctly find the robot joint angles even in the presence of numerical inaccuracies which may slightly perturb the desired pose outside the workspace.

For certain robots with the task frame placed at the wrist, the least-squares solutions from Subproblems 1–4 also solve the global least-squares problem in the usual Euclidean sense, solving the minimization

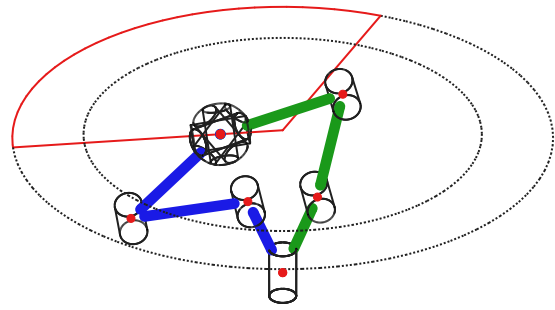
$$\min_q \|p_{0T}(q) - p_{0T}^{des}\| \quad \text{s.t. } R_{0T}(q) = R_{0T}^{des}, \quad (15)$$

where $(R_{0T}^{des}, p_{0T}^{des})$ is the desired end effector rotation and position. One robot configuration that achieves (15) is the common case of industrial robots similar to the ABB IRB 6640 [4]: a 6R robot with a spherical wrist ($p_{45} = p_{56} = 0$) and two parallel axes ($h_2 = h_3$), as well as $h_2^T(p_{12} + p_{23} + p_{34}) = 0$, $h_1^T h_2 = 0$, and $p_{6T} = 0$. We also need $h_4^T h_5 = h_5^T h_6 = 0$ so the spherical wrist can achieve any orientation. Under these conditions, we are minimizing distances in a cylindrical coordinate system. For some robots, such as when we have all the constraints above except $h_1^T h_2 \neq 0$, (15) is solved in only certain regions of the workspace.

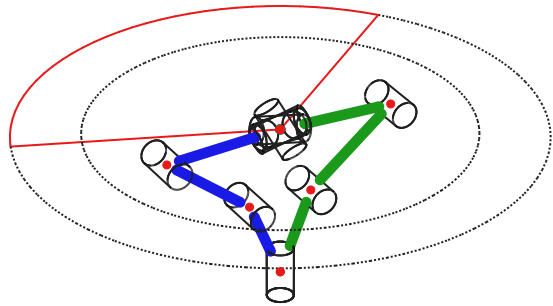
Complex-valued solutions to the IK problem are related to the continuous and least-squares IK solutions found using the subproblem decomposition method. All robots have pairs of complex conjugate solutions with joint angles $q_i = a \pm bi$ for branches that do not have real solutions. Although these solutions solve the IK problem exactly, physical robots must have real-valued joint angles. Keeping only the real part of complex-valued solutions also results in continuous approximate solutions to the IK problem. In general, these continuous approximate solutions are different from those obtained using the subproblem decomposition method, but



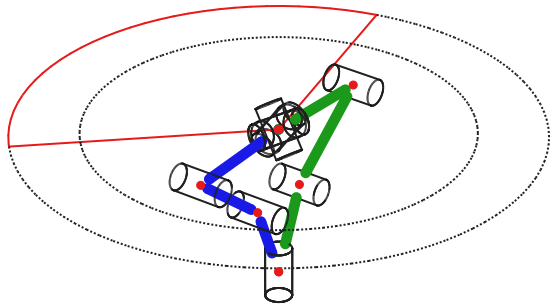
(a) $k = 40$. The desired pose is at the workspace boundary. For the shoulder forward IK solutions, the least-squares IK solutions provide numeric robustness. For the shoulder back IK solutions, the workspace boundary cannot be reached, but the least-squares IK solutions still return the closest end effector pose.



(b) $k = 90$. The desired path is inside the workspace and approaches the shoulder singularity.



(c) $k = 120$. The robot begins to rotate while in the shoulder singularity.



(d) $k = 140$. The robot continues to rotate while in the shoulder singularity.

Fig. 5. Two IK solutions over time for the ABB IRB 6640 example path. Although there are 8 branches, only the shoulder front elbow down and shoulder back elbow up branches are shown. The dotted circles denote the workspace boundary at $z = 1.5$ m for the shoulder front and shoulder back configurations. (See also Extension 1.)

there are cases where some joint angles are equal. For Subproblems 2 (with $\|p_1\| = \|p_2\|$), 3, and 4, the least-squares solution is equal to the real part of the complex solution. This is because the real part of complex solutions to a quadratic is also the least-squares real solution. For example, least-squares solutions using the subproblem decomposition method for the ABB IRB 6640 are equal to the real part of the complex solutions for q_1 or (q_1, q_2, q_3) depending on which subproblems have least-squares solutions.

To demonstrate the utility of continuous approximate and least-squares IK solutions as well as resolving a continuum of solutions, an example desired path was generated for the ABB IRB 6640 which encounters both boundary and internal singularities. The path is discretized into $K = 200$ sample points, the task frame is set to the center of the robot wrist, the orientation is constant at $R_{06} = I$, and the position is along the plane $z = 1.5$ m. Within this plane, the position moves in an arc at the workspace boundary between samples $k = 1$ through $k = 50$ then travels to the workspace center where the robot encounters a shoulder singularity at $k = 100$. Between $k = 100$ and $k = 150$ the robot undergoes self-motion. The self-motion at the shoulder singularity is parameterized by specifying not just the pose of the end effector over time but also the singular direction. This singular direction is parallel to joints 2 and 3 and can be used to find q_1 using Subproblem 4. Then, between $k = 150$ and $k = 200$ the Cartesian path returns to its starting point. In this example, the path chosen through the internal singularity means that two cycles in Cartesian space are needed to return to the original robot configuration.

All IK solutions are plotted over time in Fig. 4, and 3D plots of IK solutions over time for a few IK branches are plotted in Fig. 5 (Extension 1). Due to the shoulder offset, only half of the IK branches are able to reach the workspace boundary. This means half of the IK branches have a significant error, but they still achieve continuous motion. For the remaining branches, the IK solutions are feasible only because of the numerical stability attained by the least-squares solution. For these feasible branches, the solution oscillates between being least-squares or not depending on small numerical differences that bring the desired pose in or out of the workspace.

5. Search-based inverse kinematics

If at least one pair of consecutive joints are intersecting or parallel then the problem can be reduced to a 1D search over a circle using Subproblems 5 or 6, respectively. If a pair of nonconsecutive joints $k, k + 2$ always intersect, then the problem can also be

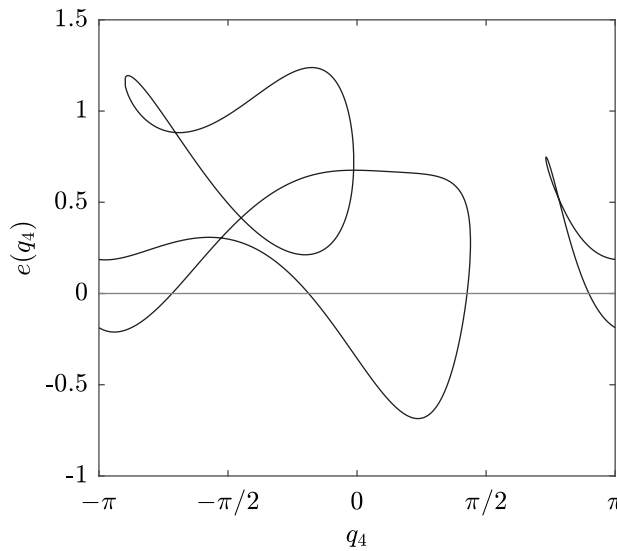


Fig. 6. Error function for a robot with axes 5 and 6 intersecting. For this end effector pose, there are four IK solutions corresponding to the four zeros of the error function. There are four branches in this error function corresponding to the four solutions of Subproblem 5. For a given input, the error function has anywhere from zero values (e.g., $q_4 = \pi/2$) to four values (e.g., $q_4 = -\pi/2$).

solved with a 1D search. If multiple pairs of joints are parallel or intersecting, then sometimes a 1D search may be performed using only Subproblems 1–4. In the fully general case where no consecutive joints are parallel or intersecting, then the problem reduces to a 2D search over a torus using Subproblem 5.

5.1. Two intersecting joint axes (1D search)

A 6R robot with two intersecting axes can be solved by searching over one joint angle. If $p_{56} = 0$ as in Fig. 3(e), then (2b) becomes

$$-p_{12} + R_{10}p_{06} = R_{12}(p_{23} + R_{23}(p_{34} + R_{34}p_{45})). \quad (16)$$

In this case, for each q_4 , we can find up to four solutions of (q_1, q_2, q_3) using Subproblem 5. The error measures the solvability of

$$R_{45}R_{56} = R_{04}^T R_{06}. \quad (17)$$

By projecting onto h_5 and h_6 , we get the error

$$e(q_4) = h_5^T R_{04}^T R_{06} h_6 - h_5^T h_6. \quad (18)$$

Perform a 1D search to find the zeros of (18), as shown in Fig. 6, and use Subproblem 1 to find q_5 and q_6 according to

$$R_{45}h_6 = R_{04}^T R_{06} h_6, \quad (19)$$

$$R_{65}h_5 = R_{06}^T R_{04} h_5. \quad (20)$$

Geometrically, we can interpret this search method as replacing the original robot with a modified robot with joint 4 fixed and an extra joint added to the intersection of joints 5 and 6. This creates a new modified robot with a spherical wrist and closed-form IK. If the IK solution for this modified robot has the new joint at zero angle, then this is a solution to the IK problem for the original robot. A similar geometric interpretation applies to any other robot with two intersecting axes: One joint angle is fixed, an extra joint is added to create a spherical joint, and this new modified robot is solved in closed form.

In the particular case of joints 5 and 6 intersecting, each point on the error function represents a robot pose where the robot wrist is at the desired position but not necessarily at the desired rotation.

5.2. Two parallel joint axes (1D search)

Since a robot with two parallel axes is the limit of a robot with two intersecting axes where the point of intersection moves infinitely far away, such a robot can also be solved with a 1D search. If $h_2 = h_3$ as in Fig. 3(f), then by substituting $R_{56} = R_{06}R_{65}$ and left multiplying by $h_2^T R_{10}$, (2b) becomes

$$h_2^T R_{10} R_{06} R_{65} p_{56} + h_2^T R_{34} p_{45} = h_2^T (R_{10} p_{06} - p_{12} - p_{23} - p_{34}). \quad (21)$$

Projecting (2a) onto h_2 and h_5 yields

$$h_2^T R_{10} R_{06} R_{65} h_5 - h_2^T R_{34} h_5 = 0. \quad (22)$$

Search over q_1 and use Subproblem 6 to find (q_4, q_6) . Next, find $q_2 + q_3$ and therefore R_{13} with Subproblem 1 to solve

$$R_{13} R_{34} h_5 = R_{10} R_{06} R_{65} h_5. \quad (23)$$

The error measures the solvability of

$$R_{12} p_{23} = R_{10} p_{06} - p_{12} - R_{13} p_{34} - R_{14} p_{45} - R_{10} R_{06} R_{65} p_{56}. \quad (24)$$

We search over q_1 to find the zeros of the error

$$e(q_1) = \|R_{10} p_{06} - p_{12} - R_{13} p_{34} - R_{14} p_{45} - R_{10} R_{06} R_{65} p_{56}\| - \|p_{23}\|. \quad (25)$$

Then solve for q_2 and q_5 with Subproblem 1, solving

$$h_2^T R_{34} R_{45} = h_2^T R_{10} R_{06} R_{65}, \quad (26)$$

$$R_{12} p_{23} = R_{10} p_{06} - p_{12} - R_{13} p_{34} - R_{14} p_{45} - R_{15} p_{56}, \quad (27)$$

and calculate q_3 with subtraction, wrapping the result to $[-\pi, \pi]$ if desired.

The geometric interpretation of this search method is similar to the case of two intersecting axes. We replace the original robot with a new modified robot with joint 1 fixed and a new prismatic joint added between joints 2 and 3. We solve IK for this modified robot in closed form, and if the new joint has zero displacement, then this is a solution to the IK problem for the original robot. A similar geometric interpretation applies to any other robot with two parallel axes.

5.3. Two nonconsecutive intersecting axes (1D search)

If a 6R robot has nonconsecutive axes k and $k+2$ always intersecting, then IK can be solved by searching over q_{k+1} in almost the same way as in Section 5.1. If axes 4 and 6 always intersect as in Fig. 3(f) then for a given q_5 we can find the point of intersection $O'_4 = O'_6$ by shifting O_4 along axis 4 and O_6 along the rotated axis 6. This gives us two new vectors p'_{04} and p'_{34} . We find (q_1, q_2, q_3) by using Subproblem 5 to solve

$$-p_{12} + R_{10} p'_{04} = R_{12}(p_{23} + R_{23} p'_{34}). \quad (28)$$

The error measures the solvability of

$$R_{03} R_{34} R_{45} R_{56} = R_{06}, \quad (29)$$

and projecting onto h_4 and h_6 yields

$$e(q_5) = h_4^T R_{03}^T R_{06} h_6 - h_4^T R_{45} h_6. \quad (30)$$

Search over q_5 to find the zeros of this error, then use Subproblem 1 to solve for q_4 and q_6 :

$$R_{34} R_{45} h_6 = R_{03}^T R_{06} h_6, \quad (31)$$

$$R_{65} R_{54} h_4 = R_{06}^T R_{03} h_4. \quad (32)$$

For some special choices of q_5 , axes 4 and 6 are parallel rather than intersecting, and the remaining joint angles must be found with Subproblem 6 rather than Subproblem 5.

5.4. General case (2D search)

IK for a general 6R robot with no intersecting or parallel axes can be solved by searching over any two joint angles. Given (q_1, q_2) , we can solve for up to four solutions of (q_3, q_4, q_5) using Subproblem 5:

$$-p_{34} + R_{32}(R_{21}(R_{10} p_{06} - p_{12}) - p_{23}) = R_{34}(p_{45} + R_{45} p_{56}). \quad (33)$$

With each $(q_1, q_2, q_3, q_4, q_5)$, compute the vector error

$$e(q_1, q_2) = R_{01} R_{12} R_{23} R_{34} R_{45} h_6 - R_{06} h_6. \quad (34)$$

Search over (q_1, q_2) to find the zeros of e . For each zero, use Subproblem 1 to solve

$$R_{56} p = R_{05}^T R_{06} p \quad (35)$$

for q_6 , where p is any vector not collinear with h_6 .

The geometric interpretation for this search method is that we replace the original robot with a new modified robot with fixed joints 1 and 2, and we add two new joints to the origin of joint 6. We solve this new robot in closed form, and if the new joints have zero angle, this is a solution to the IK problem for the original robot.

Table 4
Subproblem solution runtimes.

Subproblem	Mean runtime (ns)		
	m-file	MEX	Rust
1: Circle and point	13 517	36	59
2: Two circles	27 262	312	242
3: Circle and sphere	18 034	134	111
4: Circle and plane	17 669	105	115
5: Three circles	105 288	2 553	1 027
6: Four circles	116 750	3 110	1 436

5.5. Search implementation

For both 1D and 2D searches, the error function has multiple branches and may be undefined for some inputs, as shown in Fig. 6. Points with zero slope correspond to robot singularities, and points of infinite slope, which occur at the domain boundary for a given branch, correspond to singularities of the *modified* robot from the geometric interpretation. The slope of the 1D search error and the Jacobian of the 2D search error may be used to measure closeness to singularities.

Performing a 1D search is computationally efficient because the scalar error $e(q_i)$ crosses zero. One method is to sample the entire search space to find intervals where the error function switches sign. As long as the error function is defined and continuous throughout, zeros of the error function will be in this bracket. We can use a bracketing method such as the false position method, as summarized in [61], to efficiently find the zero. Zeros will be missed if the error function touches but does not cross zero, but this can be fixed by performing a minimization or maximization. Local minima and maxima pointing towards zero (which are locally least-squares solutions of the error function) can be returned as continuous approximate IK solutions. Zeros may also be missed if they are close to the domain boundary for some branch, but this problem can be alleviated by using least-squares or continuous approximate solutions for the subproblems when evaluating the error function.

To perform a 2D search, we can find the zeros of $\|e(q_i, q_j)\|$. The zeros are also local minima since this is a norm. We first sample the entire search space in a uniform grid and then perform numeric minimization starting with the smallest-valued sample. Once a zero is found, any neighboring samples smaller than some threshold are removed from consideration, and another minimization can be performed starting at the next smallest-valued sample until all samples below some threshold are processed.

A caveat of these particular search implementations is that certain solutions may only be found with a sufficiently small sampling interval. If we only need to return a single solution for q and have an initial guess (e.g., to find a solution closest to a previous pose), finding the zero closest to the initial guess does not require sampling the whole search space. This dramatically speeds up the computation time. Computation time can also be reduced by only searching within joint limits, and it may be useful to perform the search over joints with the smallest ranges of motion.

6. Evaluation

Subproblem and IK solutions are implemented in MATLAB and Rust. Additionally, MATLAB code has been compiled to C++ MEX code using MATLAB coder, which can be run from MATLAB. IK solutions are implemented for eight robot kinematic families. Five solutions are closed-form and return all solutions, two use 1D search and return multiple solutions with 200 initial samples, and one uses 2D search and returns multiple solutions with a grid of 100 samples in each coordinate for a total of 10 000 initial samples.

For each subproblem or IK solution, the testing code generates a file with 10 000 random test cases (or fewer for certain IK solutions), each with at least one exact solution. After loading the test cases into memory, the execution times and the returned solutions for all test cases are saved. Robot IK runtimes are measured in two ways. One way is to randomize both the robot kinematics parameters and the end effector pose. Another way is to hardcode the kinematic parameters into the program and only randomize the end effector pose. The eight hardcoded robot examples include five robots from Table 1 as well as three fictitious robots.

Timing comparisons were also made to two other publicly available analytical IK solvers: IKFast and the MATLAB Robotics Toolbox solver. IKFast generated valid code for three types of robots but failed in the other five cases. Generating and compiling code for a robot with three parallel axes and two intersecting axes (UR5) took an exceptionally long time: 55 min and 10 min, respectively. The MATLAB solver could only find solutions for two robots. The `generateIKFunction` function crashed when generating code for a robot with a spherical wrist and two intersecting axes (KUKA R800 with fixed q_3).

Testing was performed with MATLAB R2023a running on Windows 10 and rustc 1.70.0 running on Windows Subsystem for Linux with Ubuntu 18.04. The computer had an Intel Core i7-3770K CPU at 3.50 GHz and 16 GB memory.

Subproblem timing results are shown in Table 4. Subproblems 1–4 ran about an order of magnitude faster than Subproblems 5 and 6, suggesting it is advantageous to use IK algorithms that only use Subproblems 1–4. While subproblem solutions using MEX and Rust ran in under 400 ns for Subproblems 1–4 and under 4000 ns for Subproblems 5 and 6, m-file runtimes were about two orders of magnitude slower.

IK timing results are shown in Table 5. Closed-form IK solutions using the subproblem decomposition method ran in under 6 μ s for all MEX and Rust implementations, with many solutions running in under 3 μ s. Search-based IK was slower than closed-form

Table 5

Inverse kinematics solution runtimes.

Robot kinematic family	Mean random robot runtime (μ s)			Mean hardcoded robot runtime (μ s)			
	m-file (ours)	MEX (ours)	Rust (ours)	MEX (ours)	Rust (ours)	IKFast	MRT MEX
Spherical wrist	489.893	5.158	2.296	4.527	2.425	111.445	10.583
and two intersecting axes	587.320	3.423	2.062	4.991	3.688	N/A ^d	N/A ^c
and two parallel axes	745.547	3.349	2.193	2.991	3.183	4.417	13.318
Three parallel axes	391.796	5.762	3.198	5.345	4.339	N/A ^c	N/A ^c
and two intersecting axes	376.290	2.981	2.232	2.846	3.089	121.714	N/A ^c
Two intersecting axes	35 592.452 ^b	659.130	339.214	199.194	114.552	N/A ^c	N/A ^c
Two parallel axes	58 483.194 ^b	954.959	476.725	1 021.655	439.445	N/A ^c	N/A ^c
General 6R	1 490 518.240 ^a	27 904.571 ^b	18 808.249	20 741.535 ^b	16 026.010	N/A ^c	N/A ^c

^a 100 trials.^b 1000 trials.^c Did not generate code.^d Did not compile.^e Incorrect IK.

IK by a factor roughly equal to the number of initial search samples. Although m-file implementations ran slower by about two orders of magnitude, they were still fast enough for real-time control. Hardcoding the kinematics often sped up computation time for MEX code. This was most apparent for the general 6R IK since the hardcoded example was the RRC arm with fixed q_6 , meaning Subproblem 5 was solved using a quadratic rather than a quartic. Unexpectedly, Rust timing was usually slower for hardcoded kinematics. This is worth investigating as having constants and multiplications by zero should lead to faster compiled code.

MEX and Rust implementations of subproblem decomposition were faster and solved more types of robots than IKFast and Matlab Robotics Toolbox. For example, IK for a robot with three parallel axes and two intersecting axes (UR5) using subproblem decomposition was more than 40 times faster than IKFast.

7. Connection to polynomial method

Rather than using numerical search methods to find the zeros of the error functions for 6R robots without three intersecting or parallel axes, we can instead use symbolic algebra methods to convert the subproblem and error formulations into a polynomial in the tangent half-angle of one of the joints. This guarantees finding all solutions to (1), including complex solutions, to arbitrary precision. This method also allows us to detect internal and boundary singularities more easily. The method applies to other search-based solutions that use subproblem decomposition, including cases where the task is not just end effector orientation and position as for 7-DOF robots [18].

To find the polynomial, we first rewrite the subproblem decomposition and error function using the tangent half-angle substitution, which leads to three or four equations in the same number of unknowns. By expanding and clearing the denominators, we obtain a system of multivariate polynomials. The resultant univariate polynomial is found by eliminating all but one variable, and care is taken to factor out extraneous factors that may occur (such as $x^2 + 1$). After finding the zeros of this polynomial, subproblems can then be used to find the remaining joint angles in closed form. In the general 6R case, the zeros of one univariate polynomial must be substituted into an intermediate system of two variables to find solutions to two joint angles before the remaining angles are found in closed form. To improve computational efficiency, we can use rational approximations for the robot kinematics and end effector pose.

Previous literature has found polynomial solutions for the general 6R problem, but the methods presented here have important differences. These methods directly exploit intersecting or parallel axes to simplify the system of equations. Unlike some other polynomial methods, our polynomial method does not have spurious solutions.

Symbolic manipulation can be performed with Maple [62] or Mathematica [63], although Maple has better timing performance for polynomial resultants and factorization [64]. A few numerical examples are shown in Appendix B.

7.1. Errors and subproblems with tangent half-angle

The tangent half-angle substitution $x = \tan(\theta/2)$ results in

$$s_\theta = \sin(\theta) = \frac{2x}{x^2 + 1}, \quad c_\theta = \cos(\theta) = \frac{1 - x^2}{x^2 + 1}, \quad (36)$$

and the Euler–Rodrigues formula for rotations [65,66] is

$$R(k, \theta) = kk^T + s_\theta k^\times - c_\theta k^\times{}^2, \quad (37)$$

where $k^\times$ denotes the matrix representation of the cross product. For $k = [k_1 \ k_2 \ k_3]^T$,

$$k^\times := \begin{bmatrix} 0 & -k_3 & k_2 \\ k_3 & 0 & -k_1 \\ -k_2 & k_1 & 0 \end{bmatrix}, \quad (38)$$

and $k^{\times 2} = k^{\times} k^{\times} = k k^T - I$. Substituting, we get

$$R(k, \theta) = k k^T + \frac{2x}{x^2 + 1} k^{\times} + \frac{x^2 - 1}{x^2 + 1} k^{\times 2}. \quad (39)$$

When using (39) in the error or subproblem equations, it is important to realize the rotation matrix is orthogonal ($R^T R = I$) but not Hermitian in general ($R^{\dagger} R \neq I$). This means the identity $\|R p\| = \|p\|$ does not hold for complex x and θ since the norm for a complex vector is defined as $\|v\| = \sqrt{v^{\dagger} v}$. However, we can still use $p^T R^T R p = p^T p = \|p\|^2$.

We only show how to convert Subproblems 1, 4, and 6 to polynomials as they can be used to solve Subproblems 2, 3, and 5 according to the solutions in Appendix A. For completeness, the tangent half-angle solution to Subproblem 1 is

$$x = \frac{p_1^T k^{\times} p_2}{2(p_1^T k)^2 - p_1^T p_2 - p_1^T p_1}, \quad (40)$$

but the direct rotation solution described below is more computationally efficient. For Subproblem 4, substituting (39), clearing the denominators, and simplifying results in

$$(2h^T k k^T p - h^T p - d)x^2 + (2h^T k^{\times} p)x + (h^T p - d) = 0. \quad (41)$$

Similarly, for Subproblem 6, directly substituting (39) and clearing the denominators results in a pair of quadrics.

Remark 1 (Alternative Subproblem Solutions). Rather than using this polynomial conversion for robots without three parallel or intersecting axes, these conversions can be used to solve subproblems when all arguments are fixed. For Subproblem 6, we need to eliminate one variable to find a single quartic polynomial in the tangent half-angle, and the remaining angle is found by substituting back into the pair of quadrics and finding common zeros (which is equivalent to Subproblem 4).

7.2. Direct rotation solution for Subproblem 1

We can simplify the system of polynomials by directly finding $R(k, \theta)$ for Subproblem 1. Using the solutions for $\sin(\theta)$ and $\cos(\theta)$ in Appendix A along with (37), we find

$$R(k, \theta) = k k^T + \frac{(k^{\times} p_1)^T p_2}{\|k^{\times} p_1\|^2} k^{\times} + \frac{(k^{\times 2} p_1)^T p_2}{\|k^{\times} p_1\|^2} k^{\times 2}. \quad (42)$$

This can be alternatively derived by writing $R(k, \theta) = R_2 R_1^T$, where the columns of R_i are formed by normalizing $(k, k^{\times} p_i, -k^{\times 2} p_i)$. To avoid the term $\|k^{\times} p_1\| \|k^{\times} p_2\|$, as this would add extra radicals to the formulation, we use the substitution $\|k^{\times} p_1\| = \|k^{\times} p_2\|$. We can use p_1 or p_2 in the denominator depending on which one is algebraically simpler.

7.3. Inverse kinematic for two intersecting joint axes

If axes 5 and 6 intersect, the first polynomial in the system comes from the scalar error (18). Converting (16) results in two more polynomials and an expression for R_{12} which is substituted into the error equation. The result is three equations in three unknowns x_1, x_3, x_4 . The resultant can be readily found, which leads to a polynomial in x_4 . If more than two pairs of axes intersect, the error equation remains the same, but Subproblem 5 may not be needed.

Numerical examples for the FANUC CRX-10iA/L [7] and for the RRC K-1207i [10] with fixed joint 6 are shown in Appendices B.1 and B.2. Simplifications due to intersecting and parallel axes are easy to apply. For the CRX example, the system of polynomials is particularly simple in that there are very few terms and it is highly decoupled, leading to easy elimination. For the RRC example, the extra pair of nonconsecutive intersecting axes means the univariate polynomial is degree 10 rather than degree 16.

7.4. Inverse kinematics for general case

The vector error (34) is the difference of two unit vectors, meaning there are two degrees of freedom. Projecting the error twice yields two scalar errors which must equal zero and leads to the first two polynomials in the system of equations:

$$e_1 = h_3^T R_{34} R_{45} h_6 - h_3 (R_{01} R_{12})^T R_{06} h_6, \quad (43a)$$

$$e_2 = h_4^T R_{45} h_6 - h_4 (R_{01} R_{12} R_{23})^T R_{06} h_6. \quad (43b)$$

Using the Subproblem 5 conversion, (33) turns into a pair of polynomials and an expression for R_{34} , which is substituted into (43a). The result is four polynomials in four unknowns x_1, x_2, x_3, x_5 . By eliminating x_3 and x_5 , we reduce the systems to two equations in x_1, x_2 . We eliminate x_2 to find solutions for x_1 , then substitute each solution of x_1 into the pair of equations in x_1, x_2 and identify common zeros to find x_2 . The remaining joint angles are found in closed form.

There are other options for which joints to eliminate in the errors, Subproblem 5 formulation, and resultant system of polynomials. Although it is tempting to reformulate (43a) so that the rotation matrix found from Subproblem 1 is not multiplied by another rotation, this is not possible since the eliminated variables cannot be adjacent (here we consider joints 6 and 1 adjacent) to the rotation found with Subproblem 1.

Numerical examples are shown for the Husty, Pfurner, and Schröcker [25] robot and the ABB YuMi [9] with fixed joint 3 in Appendices B.3 and B.4. For the Husty, Pfurner, and Schröcker example, the robot is highly general without intersecting or parallel axes, but the subproblem decomposition immediately yields four polynomials from which two joint angles may be found. For the YuMi example, this general method is applied to a robot with a pair of nonconsecutive intersecting axes, so the resultant polynomial is degree 12 rather than degree 16.

7.5. Rational approximations and pose parameterization

Factoring and finding resultants of polynomials becomes more efficient when coefficients are rational numbers as opposed to radical expressions or symbolic sines and cosines of angles. We can find arbitrarily accurate rational approximations for $p_{i,i+1}$, h_i , and R_{06} to speed up computation, opening up a tradeoff between approximation accuracy and computational speed. Rational approximations for each p_i are straightforward, although computational efficiency is improved by maintaining cases of intersecting, orthogonal, or parallel axes or link offsets.

The rotation R_{06} only appears as multiplying another vector; for example, $R_{06}h_6$. We can parameterize R_{06} as the product of three consecutive orthogonal rotations $R_\gamma R_\beta R_\alpha$. Picking $R_\alpha = R(h_6, \alpha)$ means α does not appear in the equation. Similarly, each h_i can be written as a series of rotations applied to some basis vector. The rational approximation for R_{06} and h_i can then be reduced to finding rational approximations for $\tilde{s} \approx \sin(\theta)$ and $\tilde{c} \approx \cos(\theta)$, while maintaining the identity $\tilde{s}^2 + \tilde{c}^2 = 1$. This can be done by finding the rational approximation for $\tan(\theta/2)$ and substituting into (36) or (39). This is equivalent to finding Pythagorean triples which are the side lengths of a right triangle with one angle close to θ . Parameterizing the end effector pose in this way means the polynomial system can be found a priori in terms of p_{0T}, β, γ .

7.6. Comparison to search method

The polynomial method has several advantages over the search method. Finding the univariate polynomial is straightforward since most algebra is performed automatically. Polynomial root finding is a solved problem, so it is guaranteed to find all solutions including complex solutions, and repeated roots can be detected as robot singularities.

Despite the utility of the polynomial method, the search method remains a more practical approach. Although the system of polynomials only needs to be derived once, each new end effector pose requires symbolic manipulation to find a new resultant univariate polynomial. This means the search method is much more computationally efficient, even with the somewhat rudimentary search implementation we present here. The tangent half-angle substitution also leads to issues with joint angles around $\pm\pi$, including reduced accuracy or polynomial deflation [26], although there are strategies to overcome this singularity issue, such as by running the algorithm with two different zero configurations.

The polynomial and search methods are each useful for different reasons. Whereas the search method is fast and suitable for real-time control, the polynomial method provides stronger guarantees on the IK solutions returned and is more suited for analysis of a few end effector poses at a time.

8. Conclusion

This paper revisits the subproblem decomposition method for solving robot inverse kinematics. By converting a nonlinear problem to a linear one and then imposing constraints on the linear solutions, we found closed-form solutions to the original three Paden–Kahan subproblems as well as three additional subproblems. For Subproblems 1–4, we found the least-squares solutions in closed form when the exact problem has no solution, and we similarly found continuous approximate solutions for Subproblems 5 and 6. The inverse kinematics for any robot with three consecutive intersecting or parallel axes may be decomposed into a series of subproblems and solved directly in closed form. For other robots, the decomposition method reduces the solution to a 1D or 2D search. This method finds *all* inverse kinematics solutions instead of just one solution close to an initial guess as in Jacobian-based methods. We have also shown how to extend the subproblem decomposition method to find a high-order polynomial in the tangent half-angle of one of the joints.

We have demonstrated excellent computational performance for IK using the subproblem decomposition method, and there are still ways to speed up computation time. The implementation code for most IK solutions can be better optimized as most of the computation time is spent not on solving subproblems but on other calculations such as computing rotation matrices. Perhaps performance can be improved by computing rotations using quaternions or by directly using the sine and cosine from each subproblem solution. For search-based IK solutions, there are several possible algorithmic improvements. Sampling could be parallelized, or a much faster search method could be used, such as one that immediately starts optimization after just a few random samples, doing a random restart if that optimization fails. For 2D search, using information from each element of the vector error, rather than the norm, may also improve performance. The connection to the tangent half-angle polynomial method demonstrates that we can easily find a system of three or four polynomials, but computational performance may be improved with more efficient elimination methods. Although finding the rotation matrix directly for Subproblem 1 helps, it may be possible to find a more compact polynomial for the error equation.

Beyond 6R arms, there are other applications for the subproblem decomposition method, including the search and polynomial methods, which deserve a thorough exploration. We have already shown applications for 7R manipulators [18]. Although there have been some examples of IK for parallel manipulators and manipulators with prismatic joints such as in [17], new efficient solutions for both inverse and forward kinematics are possible.

CRediT authorship contribution statement

Alexander J. Elias: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Conceptualization. **John T. Wen:** Writing – review & editing, Supervision, Project administration, Methodology, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The authors would like to thank C. Carignan, D. Carabis, and A. Gostin for their fruitful discussions. The authors would also like to thank R. Chen, A. Maksumić, J. Mathew, and A. Ropp for their careful work in implementing the subproblem and IK solutions. Finally, the authors would like to thank the reviewers of this paper for their comprehensive comments and recommendations.

Appendix A. Subproblem solutions

Our subproblem solution method is based on geometry and linear algebra and does not rely on symbolic manipulation. To improve precision and prevent edge cases where the solution method fails, ATAN2 is the only inverse trigonometric function used to as it is extremely robust compared to $\sin^{-1}$ and $\cos^{-1}$ [12], as well as compared to solving for $\tan(\theta/2)$ [24,26]. Subproblems 2, 3, and 5 are solved using Subproblem 4. Subproblems 5 and 6 are reduced to solving the intersection of two ellipses, which may be solved by finding the roots of a quartic polynomial. In Subproblems 1–4, if an exact solution does not exist, then we instead find the solution which minimizes Euclidean distance. Conveniently, this least-squares solution requires almost no overhead to compute. The minimizing solutions between two smooth objects occurs on a line normal to both objects. As pointed out in [67], for a circle the common normal passes through the circle axis. The continuous approximate solutions for Subproblems 5 and 6 are found by using all solutions of the quartic polynomial including complex conjugate pairs of solutions. Complex numbers are used up until ATAN2, where only the real part is used. To better identify singularities and to increase computational efficiency, we explain conditions under which the subproblems may be decomposed further.

We solve Subproblems 1, 4, and 6 by writing them as a linear equations in $x = [s_\theta \ c_\theta]^T$, where $s_\theta = \sin \theta$ and $c_\theta = \cos \theta$, with nonlinear constraint $\sin^2(\theta) + \cos^2(\theta) = \|x\|^2 = 1$. We first solve the unconstrained linear problem with the tools of linear algebra, and then find the subset of solutions following the constraint. A similar approach was used in [68] to find intersections among spheres. To convert a subproblem to a linear problem, we rewrite (37) as

$$R(k, \theta)p = p_k + A_{k,p}x, \quad (\text{A.1})$$

where

$$p_k = k k^T p \quad \text{and} \quad A_{k,p} = \begin{bmatrix} k^\times p & -k^\times{}^2 p \end{bmatrix}. \quad (\text{A.2})$$

Geometrically, this conversion replaces any circle with the plane in which the circle lies. The vector p_k is along the circle axis pointing to the circle center, and the columns of $A_{k,p}$ form an orthogonal basis for the plane and have norm equal to the circle radius.

Given $x = [s_\theta \ c_\theta]^T$, the corresponding angle is found using $\theta = \text{ATAN2}(s_\theta, c_\theta)$. We can achieve some computational speed-up by recognizing $\text{ATAN2}(s_\theta, c_\theta) = \text{ATAN2}(\alpha s_\theta, \alpha c_\theta)$ where $\alpha > 0$. This means we only have to solve for αx .

Subproblem 1: Circle and point

Given vectors p_1, p_2 and a unit vector k , find θ to minimize $\|R(k, \theta)p_1 - p_2\|$.

The solution is a continuum, i.e., θ is arbitrary, when (k, p_1) or (k, p_2) are collinear. Otherwise,

$$\theta = \text{ATAN2}((k^\times p_1)^T p_2, -(k^\times{}^2 p_1)^T p_2). \quad (\text{A.3})$$

The solution is exact when $\|p_1\| = \|p_2\|$ and $k^T p_1 = k^T p_2$.

The derivation of (A.3) is as follows. Using (A.1), we have

$$\|R(k, \theta)p_1 - p_2\| = \|A_{k,p_1}x - b\|, \quad (\text{A.4})$$

where $b = p_2 - p_{1k} = p_2 - k k^T p_1$. If (k, p_1) and (k, p_2) are not collinear then A_{k,p_1} has full column rank. An exact solution to $A_{k,p_1}x = b$ such that $\|x\| = 1$ exists when p_2 lies on the circle, meaning $\|p_1\| = \|p_2\|$ and $k^T p_1 = k^T p_2$. In this case, x is the unconstrained least-squares solution

$$x_{LS} = \min_x \|A_{k,p_1}x - b\| = A_{k,p_1}^+ b = A_{k,p_1}^+ p_2, \quad (\text{A.5})$$

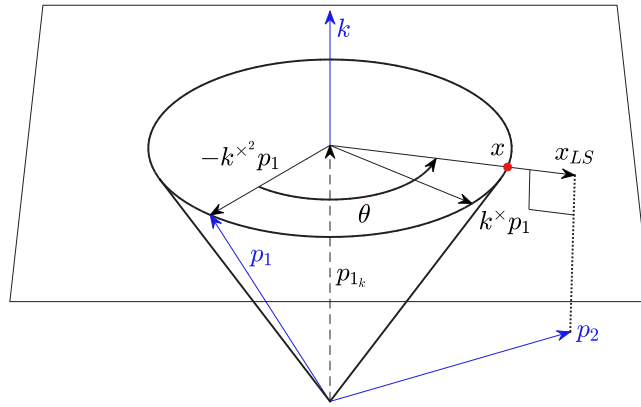


Fig. A.7. Subproblem 1 finds the angle of the closest point on the circle $R(k, \theta)p_1$ to the point p_2 . This is the angle of x_{LS} in the coordinate frame in the circle plane formed by $(-k^{\times 2}p_1, k^{\times}p_1)$, where x_{LS} is the projection of p_2 to the circle plane.

where $A^+ = (A^T A)^{-1} A^T$ is the pseudo-inverse (left inverse) of A . One can show $A_{k,p_1}^+ = A_{k,p_1}^T / \|k^{\times} p_1\|^2$.

In general, an exact solution is not possible, and we need to solve the constrained least-squares problem

$$\min_x \|A_{k,p_1} x - b\| \quad \text{s.t.} \quad \|x\| = 1. \quad (\text{A.6})$$

Since the common normal between the circle and p_2 must pass through the circle's axis, the minimizing solution occurs on the plane of symmetry containing the circle's axis and p_2 . As shown in Fig. A.7, x_{LS} is the coordinates in the coordinate system $(-k^{\times 2}p_1, k^{\times}p_1)$ of p_2 projected onto the plane containing the circle. This vector is in the plane of symmetry, and normalizing it gives the constrained least-squares solution $x = x_{LS} / \|x_{LS}\|$. Skipping all positive scalar division (which keeps the value of ATAN2 unaffected) yields the general solution (A.3).

Note that all other p_2 vectors which lie in the same half plane containing k result in the same least-squares solution.

Subproblem 2: Two circles

Given vectors p_1, p_2 and unit vectors k_1, k_2 , find (θ_1, θ_2) to minimize $\|R(k_1, \theta_1)p_1 - R(k_2, \theta_2)p_2\|$.

The solution is exact if $\|p_1\| = \|p_2\|$, as this means the two circles lie on the same sphere. We see the minimizing (θ_1, θ_2) is independent of $\|p_1\|$ and $\|p_2\|$ if we rewrite the subproblem as

$$\|R(k_1, \theta_1)p_1 - R(k_2, \theta_2)p_2\|^2 = \|p_1\|^2 + \|p_2\|^2 - 2\|p_1\|\|p_2\|\cos(\psi), \quad (\text{A.7})$$

where ψ is the angle between the rotated vectors. This means we can find the least-squares solution(s) by normalizing p_1 and p_2 before proceeding. Then, the solutions are exact if and only if

$$|\theta_{k_1,p_1} - \theta_{k_2,p_2}| \leq \theta_{k_1,k_2} \leq \theta_{k_1,p_1} + \theta_{k_2,p_2}, \quad (\text{A.8})$$

where θ_{v_1,v_2} is the angle between vectors v_1 and v_2 . This inequality can be derived by considering the intersections of the circles with the plane of symmetry spanned by k_1 and k_2 . A solution exists if and only if the intersections are interleaved, that is, the intersections alternate between circles 1 and 2 as we rotate along the plane of symmetry. If p_1 is arbitrary, then θ_{k_1,p_1} can be any angle from 0 to π , and a solution always exists if and only if $k_1^T k_2 = 0$ and $k_2^T p_2 = 0$. This condition (A.8) is checked implicitly when solving with Subproblem 4 after projecting onto k_2 and k_1 :

$$k_2^T R(k_1, \theta_1)p_1 = k_2^T p_2, \quad (\text{A.9a})$$

$$k_1^T R(k_2, \theta_2)p_2 = k_1^T p_1. \quad (\text{A.9b})$$

If there are two solutions for θ_1 and θ_2 , ensure the solutions match by switching the order of solutions for θ_2 . If θ_1 and θ_2 have different numbers of solutions due to numerical issues, duplicate the solution for the angle with only one solution.

The solution is a continuum if (k_1, p_1) , (k_2, p_2) , or (k_1, k_2) are collinear, which are conditions checked by Subproblem 4. In each case, θ_1 , θ_2 , or $\theta_1 \pm \theta_2$ is arbitrary, respectively.

Subproblem 4 finds the intersections between each circle and the plane in which the other circle lies, as shown in Fig. A.8(a). Since the two circles lie on the same sphere, and there are no extraneous solutions: each intersection between a circle and the other plane must also be an intersection with the other circle.

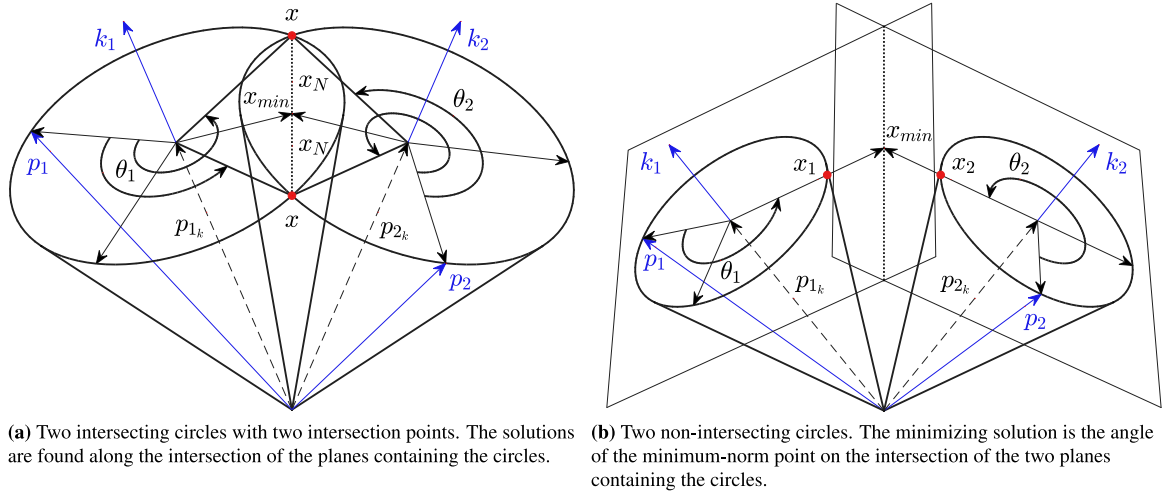


Fig. A.8. Subproblem 2 finds the angles of the closest points on the two circles $R(k_1, \theta_1)p_1$ and $R(k_2, \theta_2)p_2$.

If the two circles do not intersect, as in Fig. A.8(b), the least-squares solution is also given by Subproblem 4, and this case can be detected by checking if Subproblem 4 returns a least-squares result. The minimizing solutions occur when line passing through $R(k_1, \theta_1)p_1$ and $R(k_2, \theta_2)p_2$ passes through both circle axes, meaning the solutions must occur on the plane of symmetry containing k_1 and k_2 . (We have removed the case of the line being out of plane but passing through the origin by assuming $\|p_1\| = \|p_2\|$.) The least-squares solution for Subproblem 4 lies on this same plane.

Remark 2 (Alternate Decomposition). Alternatively, solve Subproblem 4 for one angle, then use Subproblem 1 to find the remaining angle solutions.

Remark 3 (Linear Solution). This subproblem may be solved by rewriting the problem as a linear equation $Ax = b$, where

$$A = [A_{k_1, p_1} \quad A_{k_2, p_2}] \quad \text{and} \quad b = p_{2k} - p_{1k}. \quad (\text{A.10})$$

The complete solution $x = x_{min} + x_N$ is the intersection between the two planes, where x_{min} is the minimum-norm solution and x_N is an arbitrary vector in the one-dimensional null space of A . Simplifying to find efficient expressions for x such that $\|x_i\| = 1$ results in the solution above.

Remark 4 (Skew Axes). It is straightforward to extend this subproblem to the case where the circle axes are skew, solving

$$p_0 + R(k_1, \theta_1)p_1 = R(k_2, \theta_2)p_2. \quad (\text{A.11})$$

The solution is nearly identical: Project onto k_1 or k_2 and solve Subproblem 4 and Subproblem 1 without normalizing p_1 and p_2 . Since the axes are skew, there is only up to one solution, and we need to check for extraneous solutions by checking if both subproblems returned an exact solution. This problem can be solved in a similar way using Subproblem 3 rather than Subproblem 4 by taking the norm of both sides of the equation. Solving the least-squares version of this problem is possible but complicated as there are up to eight common normals between two circles [69].

When the circle axes are skew, the two circles lie on different spheres, and the problem is generally unsolvable: with randomly chosen parameters, there is almost surely no solution for (θ_1, θ_2) . It is therefore inadvisable to use this as part of the solution to a larger geometry problem.

Remark 5 (Intersecting Axes). If p_0 can be written as a linear combination of k_1 and k_2 , then the circle axes still intersect, and the problem can be solved with Subproblem 2 by moving the respective components of p_0 to p_1 and p_2 .

Remark 6 (Parallel Axes). If the circle axes are parallel ($k_1 = k_2 = k$), we require $k^T(p_0 + p_1 - p_2) = 0$ to guarantee exact solutions. Up to two solutions of θ_1 may be solved using Subproblem 3, and the corresponding solutions of θ_2 may be found using Subproblem 1. We can also find up to two least-squares solutions if $k^T(p_0 + p_1 - p_2) \neq 0$ by replacing p_i with $-k^{\times 2}p_i$ for $i = 0, 1, 2$.

Subproblem 3: Circle and sphere

Given vectors p_1 , p_2 , a unit vector k , and a nonnegative scalar d , find θ to minimize $\|R(k, \theta)p_1 - p_2\| - d$.

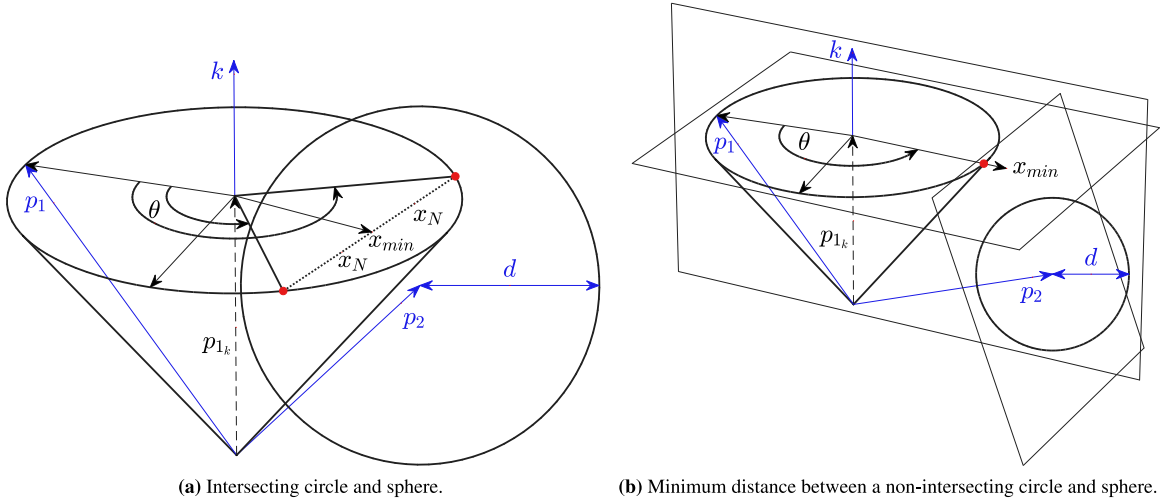


Fig. A.9. Subproblem 3 finds the angles of the closest points on the circle $R(k, \theta)p_1$ to the sphere of radius d centered at p_2 .

Rewrite equivalently and solve with Subproblem 4:

$$\left| \frac{1}{2} \|R(k, \theta)p_1 - p_2\|^2 - \frac{d^2}{2} \right| = \left| p_2^T R(k, \theta)p_1 - \frac{1}{2} (\|p_1\|^2 + \|p_2\|^2 - d^2) \right|. \quad (\text{A.12})$$

The solution is a continuum if (k, p_1) or (k, p_2) are collinear, but this condition is checked by Subproblem 4.

If the circle and sphere intersect, as in Fig. A.9(a), then this conversion, which is basically the law of cosines, replaces the sphere with a plane which intersects the circle at the same points. If the circle and sphere do not intersect, as in Fig. A.9(b), the minimizing angle on the circle to the plane is also the minimizing angle to the sphere, which occurs on the line passing through the circle axis and the center of the sphere. Interestingly, if $d = 0$ then this subproblem solves Subproblem 1.

Subproblem 4: Circle and plane

Given a vector p , unit vectors k , h , and a scalar d , find θ to minimize $|h^T R(k, \theta)p - d|$.

The solution is a continuum if (p, k) or (h, k) are collinear. Else, define

$$x_{\min} = \frac{A^T}{\|A^T\|^2} b, \quad (\text{A.13})$$

where $A = h^T A_{k,p}$ and $b = d - h^T k k^T p$. If $\|x_{\min}\| < 1$,

$$x = x_{\min} \pm \frac{\sqrt{\|A^T\|^2 - b^2}}{\|A^T\|^2} \underbrace{\begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}}_J A^T. \quad (\text{A.14})$$

If $\|x_{\min}\| \geq 1$, $x = x_{\min}$ and the solution is least-squares.

The derivation of this solution is as follows. If we apply (A.1), then $h^T R(k, \theta)p - d = Ax - b$, where $A = h^T A_{k,p}$ and $b = d - h^T k k^T p$. There are many solutions to $Ax = b$, which can be parameterized as $x = A^+ b + x_N$, where $A^+ = A^T (AA^T)^{-1}$ (for A with full row rank) is the pseudo-inverse (right inverse) of A , $x_{\min} = A^+ b$ is the minimum-norm solution, and x_N is an arbitrary vector in the null space of A , which is orthogonal to $x_{\min}$. If (p, k) and (h, k) are not collinear, the 2×1 row vector A is of rank one and has a one-dimensional null space given by $x_N = \xi x'_N$, where $x'_N = J A^T$, x'_N forms a basis for the null space, and ξ parameterizes the solution. We can use the $\|x\|^2 = 1$ constraint to find ξ as the solution to a quadratic equation. The null space is the intersection between the plane containing the circle and a plane perpendicular to h . The minimum-norm solution $x_{\min}$ is given by (A.13) which is the point of intersection among three planes: The plane containing the circle, the plane perpendicular to dh , and the plane of symmetry spanned by k and h .

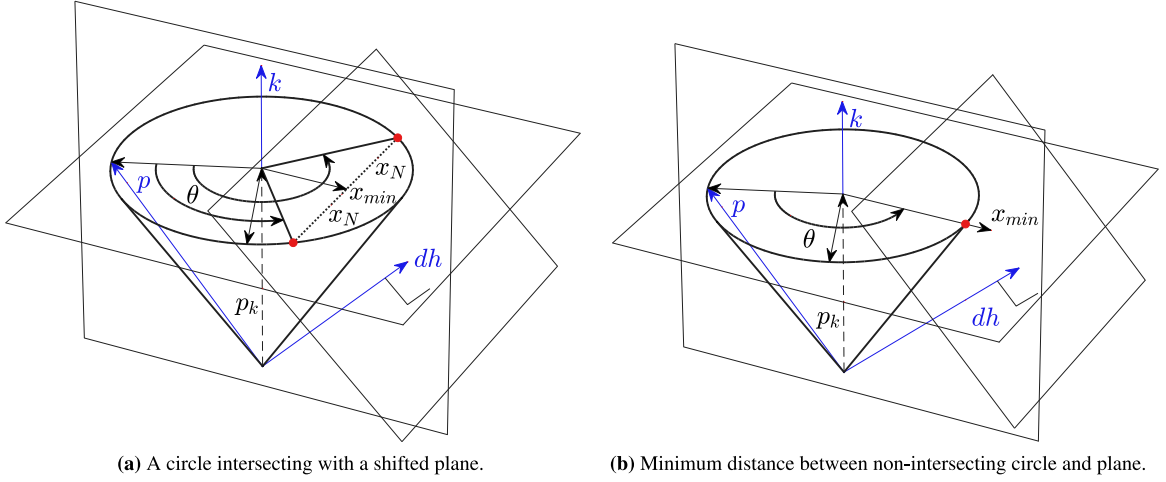


Fig. A.10. Subproblem 4 finds the angles of the closest points on the circle $R(k, \theta)p$ to the plane with normal vector h and distance d from the origin.

If $\|x_{min}\| < 1$, as in Fig. A.10(a), two intersections occur, x_{min} corresponds to an interior point of the circle, and the two choices for ξ are

$$\xi = \pm \frac{\sqrt{1 - \|x_{min}\|^2}}{\|x'_N\|} = \pm \frac{\sqrt{\|A^T\|^2 - b^2}}{\|A^T\|^2}. \quad (\text{A.15})$$

If $\|x_{min}\| = 1$, one intersection occurs, and $x = x_{min}$. If $\|x_{min}\| > 1$, as in Fig. A.10(b), the circle does not intersect with the plane. The minimizing solution occurs on the common normal which is perpendicular to the plane and passes through the circle axis. This means the solution occurs on the plane of symmetry, and x is therefore the normalized version of x_{min} . Normalization can be skipped since x is evaluated by ATAN2.

If h is not a unit vector, the solution method stays the same. The only difference is in the geometric interpretation.

Subproblem 5: Three circles

Given vectors, p_0, p_1, p_2, p_3 and unit vectors k_1, k_2, k_3 , find $(\theta_1, \theta_2, \theta_3)$ to solve

$$p_0 + R(k_1, \theta_1)p_1 = R(k_2, \theta_2)(p_2 + R(k_3, \theta_3)p_3). \quad (\text{A.16})$$

We may visualize this subproblem as the intersection of three circles, as depicted in Fig. A.11:

- Circle 1 is given by $p_0 + R(k_1, \theta_1)p_1$.
- Circle 2 has axis k_2 . Its radius r and height z are to be determined, where z may be negative if the circle center in the $-k_2$ direction.
- Circle 3 is given by $p_2 + R(k_3, \theta_3)p_3$.

The solution is a continuum if (k_1, p_1) or (k_3, p_3) are collinear. Other cases for a continuum of solutions exist if $p_0^T k_1^\times k_2 = 0$ or $p_2^T k_3^\times k_2 = 0$. Write

$$\begin{aligned} q_1 &= p_0 + R(k_1, \theta_1)p_1 = A_{k_1, p_1} x_1 + p_{1_S}, \\ q_3 &= p_2 + R(k_3, \theta_3)p_3 = A_{k_3, p_3} x_3 + p_{3_S}, \end{aligned} \quad (\text{A.17})$$

where the shifted centers of the circles are

$$p_{1_S} = p_0 + k_1 k_1^T p_1, \quad p_{3_S} = p_2 + k_3 k_3^T p_3. \quad (\text{A.18})$$

The projection of the solutions on circle 1 and circle 3 to k_2 is z :

$$z = k_2^T (p_0 + R(k_1, \theta_1)p_1) = k_2^T (p_2 + R(k_3, \theta_3)p_3). \quad (\text{A.19})$$

Use Subproblem 4 to find x_1 and x_3 in terms of z :

$$x_i = \frac{v_i(z - \delta_i) \pm J v_i \sqrt{\|v_i\|^2 - (z - \delta_i)^2}}{\|v_i\|^2}, \quad (\text{A.20})$$

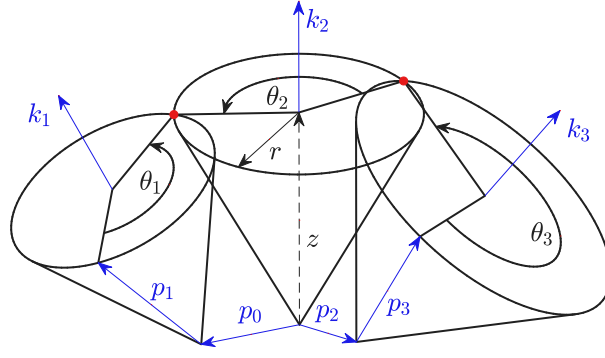


Fig. A.11. Subproblem 5 finds the angles of intersections among three circles, where radius r and height z of circle 2 are to be determined and depend on θ_1 and θ_3 . In the scenario depicted here, there exist other solutions of $(\theta_1, \theta_2, \theta_3)$ associated with different (r, z) .

where J is defined in (A.14), $v_i = A_{k_i p_i}^T k_2$, and $\delta_i = k_2^T p_{iS}$. Substituting, we find the two ellipse equations, which are distorted versions of the cross sections of the surfaces of revolution of circles 1 and 3 around k_2 :

$$q_i^T q_i = \|k_i^\times p_i\|^2 + \|p_i\|^2 + 2\alpha_i \delta_i - 2\alpha_i z \pm 2\beta_i \sqrt{\|k_i^\times p_i\|^2 \|k_i^\times k_2\|^2 - (z - \delta_i)^2}, \quad (\text{A.21})$$

where

$$\alpha_i = \frac{p_{i-1}^T k_i^\times k_2}{\|k_i^\times k_2\|^2} \text{ and } \beta_i = \frac{p_{i-1}^T k_i^\times k_2}{\|k_i^\times k_2\|^2}. \quad (\text{A.22})$$

To arrive at (A.21), we have used

$$A_{k_i p_i}^T J A_{k_i p_i}^T = \|k_i^\times p_i\|^2 k_i^\times, \quad (\text{A.23})$$

$$\|A_{k_i p_i}^T k_2\|^2 = \|k_i^\times p_i\|^2 \|k_i^\times k_2\|^2, \quad (\text{A.24})$$

$$A_{k_i p_i} A_{k_i p_i}^T = -\|k_i^\times p_i\|^2 k_i^{\times 2}. \quad (\text{A.25})$$

Equating the two right-hand sides of (A.21) results in an equation of the form $P_1 \pm \sqrt{R_1} = P_3 \pm \sqrt{R_3}$, and isolating and squaring for each radical yields $(P_1 - P_3)^4 - 2(P_1 - P_3)^2(R_1 + R_3) + (R_1 - R_3)^2 = 0$, a quartic polynomial in z . (In certain cases, it degenerates to a lower-degree polynomial.) There are many ways to find the roots of a quartic polynomial in closed form [70]. For each root z_i , find solutions for x_1 and x_3 by computing the positive and negative branches in (A.20) and checking for $q_1^T q_1 = q_3^T q_3$. Then, find θ_2 using Subproblem 1 to solve (A.16) using the value from (A.17). As two ellipses can intersect up to four times, there are up to four solutions for $(\theta_1, \theta_2, \theta_3)$.

Remark 7 (Coplanar Axes). The solution procedure simplifies when $p_0^T k_1^\times k_2 = 0$ or $p_2^T k_3^\times k_2 = 0$, meaning axis 1 or 3 is coplanar with axis 2. In this case, the equation for one of the ellipses degenerates into a line segment, and revolving circles 1 or 3 around k_2 results in a segment of a sphere or plane. If two axes intersect, rewrite the subproblem such that $p_0 = 0$ or $p_2 = 0$ and use Subproblem 3 and then Subproblem 2 to find up to four solutions. If two axes are parallel, use Subproblem 4 and then Subproblem 3 and Subproblem 1.

Remark 8 (Decomposition to Subproblem 6). As was done in [26], taking half the squared norm of both sides of (A.16) yields

$$p_0^T R(k_1, \theta_1) p_1 - p_2^T R(k_3, \theta_3) p_3 = \frac{1}{2} (\|p_2\|^2 + \|p_3\|^2 - \|p_0\|^2 - \|p_1\|^2), \quad (\text{A.26})$$

and projecting (A.16) onto k_2 yields

$$k_2^T R(k_1, \theta_1) p_1 - k_2^T R(k_3, \theta_3) p_3 = k_2^T p_2 - k_2^T p_0. \quad (\text{A.27})$$

These two equations may be solved with Subproblem 6. Then, θ_2 can be solved with Subproblem 1.

Subproblem 6: Four circles

Given vectors p_i , unit vectors k_i , h_i , $i = 1, \dots, 4$, and scalars d_1, d_2 , find (θ_1, θ_2) to solve

$$\begin{cases} h_1^T R(k_1, \theta_1) p_1 + h_2^T R(k_2, \theta_2) p_2 = d_1 \\ h_3^T R(k_3, \theta_1) p_3 + h_4^T R(k_4, \theta_2) p_4 = d_2. \end{cases} \quad (\text{A.28})$$

This problem involves four circles defined by (k_i, p_i) , where each circle also has a direction vector h_i . The position on the circle gets projected onto the h_i direction to get a distance for that circle. These circles are coupled so that the sums of distances for circles (1, 2) and (3, 4) must equal (d_1, d_2) , and also coupled so that circles (1, 3) and (2, 4) have the same rotation angles (θ_1, θ_2) .

Use (A.1) to rewrite the subproblem as

$$\begin{bmatrix} h_1^T A_{k_1, p_1} & h_2^T A_{k_2, p_2} \\ h_3^T A_{k_3, p_3} & h_4^T A_{k_4, p_4} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = Ax = b = \begin{bmatrix} d_1 - h_1^T k_1 k_1^T p_1 - h_2^T k_2 k_2^T p_2 \\ d_2 - h_3^T k_3 k_3^T p_3 - h_4^T k_4 k_4^T p_4 \end{bmatrix}. \quad (\text{A.29})$$

If row 1 is a multiple of row 2, then A loses rank and there is a continuum of solutions. In this case, pick any nonzero equation from (A.28), choose θ_1 or θ_2 arbitrarily (within a valid range), and solve for the other joint angle with Subproblem 4.

If A is full rank, the unconstrained solutions to (A.29) are

$$x = x_{\min} + \xi_1 x'_{N1} + \xi_2 x'_{N2}, \quad (\text{A.30})$$

where $x_{\min} = A^+ b$ is the minimum-norm solution, $x_N = \xi_1 x'_{N1} + \xi_2 x'_{N2}$ is an arbitrary vector in the two-dimensional null space of A , x'_{N_i} forms a basis for the null space, and ξ_i parameterizes the solution. The values of $x_{\min}$, x'_{N1} , and x'_{N2} can be found by QR decomposition of A^T .

If we impose the constraints $\|x_1\| = 1$ and $\|x_2\| = 1$, we get two equations for ellipses in (ξ_1, ξ_2) . Like in Subproblem 5, the intersection of two ellipses can be found by finding the roots of a quartic polynomial. However, in this subproblem, the equations for the ellipses are in standard form, so some extra work is needed to find the quartic equation [71]. The solutions for (ξ_1, ξ_2) can then be used to find solutions to (x_1, x_2) and therefore the two unknown angles.

As in Subproblem 4, this solution method still works even if h_i are not unit vectors.

Remark 9 (Simplified Case). The problem simplifies if h_i or p_i is parallel to k_i for some i . In this case, one of the equations in (A.28) reduces to Subproblem 4. After solving for one angle from that equation, the other equation can be solved by using Subproblem 4 again.

Appendix B. Polynomial method numerical examples

We provide four examples of the polynomial method described in Section 7.

B.1. Three pairs of intersecting axes: FANUC CRX-10iA/L

The FANUC CRX-10iA/L [7] has kinematic parameters

$$p_{12} = p_{34} = p_{56} = 0, \quad p_{23} = 710e_z, \quad p_{45} = 540e_y + 150e_z, \quad h_1 = e_z, \quad h_2 = h_3 = h_5 = e_x, \quad h_4 = h_6 = e_y, \quad (\text{B.1})$$

where e_x , e_y , e_z are the standard basis vectors. For this example, pick $R_{06} = I$ and $p_{06} = 250e_x + 250e_y + 250e_z$. We get one equation for the error, one equation for Subproblem 3, and two equations for Subproblem 4:

$$\frac{2x_4(x_1^2 - 1)(-x_2^2 x_3 - x_2 x_3^2 + x_2 + x_3)}{(x_1^2 + 1)(x_2^2 + 1)(x_3^2 + 1)(x_4^2 + 1)} + \frac{x_1(x_4^2 - 1)}{(x_1^2 + 1)(x_4^2 + 1)} = 0, \quad (\text{B.2a})$$

$$\frac{x_1^2 x_4^2 - 5x_1 x_4^2 + 4x_1^2 - 4x_4^2 - 5x_1 - 1}{(x_4^2 + 1)} = 0, \quad (\text{B.2b})$$

$$\begin{aligned} & \frac{3x_2^2 x_4(x_3^2 - 1)}{2(x_3^2 + 1)(x_4^2 + 1)} + \frac{27x_2^2 x_3}{5(x_3^2 + 1)} + \frac{24x_2^2}{5} + \frac{27x_2(x_3^2 - 1)}{5(x_3^2 + 1)} \\ & - \frac{6x_2 x_3 x_4}{(x_3^2 + 1)(x_4^2 + 1)} - \frac{3x_4(x_3^2 - 1)}{2(x_3^2 + 1)(x_4^2 + 1)} - \frac{27x_3}{5(x_3^2 + 1)} - \frac{23}{10} = 0, \end{aligned} \quad (\text{B.2c})$$

$$\frac{1}{(x_4^2 + 1)}(6307x_3^2 x_4^2 + 4260x_3^2 x_4 + 15336x_3 x_4^2 + 6307x_3^2 + 6307x_4^2 + 15336x_3 - 4260x_4 + 6307) = 0. \quad (\text{B.2d})$$

Clearing the denominators, we get

$$\begin{aligned} P_1 = & x_1 x_2^2 x_3^2 x_4^2 - 2x_1^2 x_2^2 x_3 x_4 - 2x_1^2 x_2 x_3^2 x_4 - x_1 x_2^2 x_3^2 + x_1 x_2^2 x_4^2 + x_1 x_3^2 x_4^2 + 2x_1^2 x_2 x_4 \\ & + 2x_1^2 x_3 x_4 + 2x_2^2 x_3 x_4 + 2x_2 x_3^2 x_4 - x_1 x_2^2 - x_1 x_3^2 + x_1 x_4^2 - 2x_2 x_4 - 2x_3 x_4 - x_1 = 0, \end{aligned} \quad (\text{B.3a})$$

$$P_2 = x_1^2 x_4^2 - 5x_1 x_4^2 + 4x_1^2 - 4x_4^2 - 5x_1 - 1 = 0, \quad (\text{B.3b})$$

$$\begin{aligned}
P_3 = & 48x_2^2x_3^2x_4^2 + 54x_2^2x_3x_4^2 + 15x_2^2x_3^2x_4 + 54x_2x_3^2x_4^2 + 48x_2^2x_3^2 + 48x_2^2x_4^2 \\
& - 23x_3^2x_4^2 + 54x_2^2x_3 - 15x_2^2x_4 + 54x_2x_3^2 - 60x_2x_3x_4 - 54x_2x_4^2 \\
& - 15x_3^2x_4 - 54x_3x_4^2 + 48x_2^2 - 23x_3^2 - 23x_4^2 - 54x_2 - 54x_3 + 15x_4 - 23 = 0,
\end{aligned} \tag{B.3c}$$

$$P_4 = 6307x_3^2x_4^2 + 4260x_3^2x_4 + 15336x_3x_4^2 + 6307x_3^2 + 6307x_4^2 + 15336x_3 - 4260x_4 + 6307 = 0. \tag{B.3d}$$

These equations are highly decoupled. P_1 depends on (x_1, x_2, x_3, x_4) , P_2 on (x_1, x_4) , P_3 on (x_2, x_3, x_4) , and P_4 on (x_3, x_4) . We can eliminate three variables by only taking resultants three times. First, we combine (P_1, P_2) to eliminate x_1 . Then, we eliminate x_2 by combining with P_3 . Finally, we eliminate x_3 by combining with P_4 . The factor of interest is the order 16 polynomial in x_4 :

$$\begin{aligned}
& 592628990976x_4^{16} - 493649640000x_4^{15} - 7228671199758x_4^{14} - 8539013655000x_4^{13} - 2994570828222x_4^{12} \\
& + 2896653453750x_4^{11} + 45929194539844x_4^{10} + 39808558518750x_4^9 + 177776490417945x_4^8 \\
& - 39808558518750x_4^7 + 45929194539844x_4^6 - 2896653453750x_4^5 - 2994570828222x_4^4 \\
& + 8539013655000x_4^3 - 7228671199758x_4^2 + 493649640000x_4 + 592628990976 = 0.
\end{aligned} \tag{B.4}$$

The eight real roots of this polynomial correspond to the eight IK solutions for this pose.

B.2. Two intersecting axes: RRC K-1207i with fixed joint 6

For this robot [10], we fix $q_6 = \pi/6$ and choose the pose

$$R_{06} = R(e_y, \pi/6)R(e_z, \pi/9)R(h_7, \pi/18), \tag{B.5a}$$

$$p_{06} = 30.12e_x + 20.34e_y + 20.56e_z, \tag{B.5b}$$

to find three equations in (x_1, x_3, x_4) , where one equation has 200 terms and two equations each have 16 terms. We eliminate x_1 to get two equations with 241 and 10 terms. Then, we eliminate x_3 to get one equation with 75 terms. Factoring, we get a polynomial of degree 10 with six real solutions for x_4 .

B.3. General 6R: Husty, Pfurner, and Schröcker robot

We will apply the polynomial method to a highly general robot described in [25]. The nominal kinematic parameters are

$$\begin{aligned}
p_{12} &= 10e_x + 100e_z, \quad p_{23} = 100e_x + 25e_y + 25\sqrt{3}e_z, \quad p_{34} = 150e_x + 50e_z, \\
p_{45} &= 20e_x + 25\sqrt{3}e_y - 25e_z, \quad p_{56} = 5e_z + 10e_y - 3\sqrt{3}e_z, \\
h_1 &= h_3 = h_6 = e_z, \quad h_2 = \frac{1}{2}e_y + \frac{\sqrt{3}}{2}e_z, \quad h_4 = -\frac{\sqrt{3}}{2}e_y + \frac{1}{2}e_z, \quad h_5 = -\frac{1}{2}e_y + \frac{\sqrt{3}}{2}e_z.
\end{aligned} \tag{B.6}$$

Applying the rational approximation with $\theta = \pi/6$ gives $\tilde{c} = 564719/652081$, $\tilde{s} = 326040/652081$, and the approximate kinematic parameters

$$\tilde{h}_2 = \tilde{s}e_y + \tilde{c}e_z, \quad \tilde{h}_4 = -\tilde{c}e_y + \tilde{s}e_z, \quad \tilde{h}_5 = -\tilde{s}e_y + \tilde{c}e_z. \tag{B.7}$$

We can similarly approximate the position vectors:

$$\tilde{p}_{23} = 100e_x + 25e_y + \frac{23859}{551}e_z, \quad \tilde{p}_{45} = 20e_x + \frac{23859}{551}e_y - 25e_z, \quad \tilde{p}_{56} = 5e_x + 10e_y - \frac{46367}{2677}e_z. \tag{B.8}$$

The problem is to find all IK solutions corresponding to the end effector pose given by the forward kinematics (based on the nominal kinematics parameters) of the joint angles

$$q = \left[-\frac{\pi}{6} \quad \frac{\pi}{2} \quad -\frac{\pi}{3} \quad \frac{\pi}{2} \quad \frac{\pi}{6} \quad -\frac{\pi}{6} \right]^T. \tag{B.9}$$

The desired pose is approximated as

$$R_{06} = R(e_z, \gamma)R(e_y, \beta)R(e_z, \alpha), \quad \tan(\tilde{\beta}/2) = \frac{571}{1284}, \quad \tan(\tilde{\gamma}/2) = -\frac{2858}{871}, \quad \tilde{p}_{06} = \frac{73288}{345}e_x + \frac{33409}{245}e_y + \frac{655}{97}e_z. \tag{B.10}$$

Applying the subproblem decomposition, we have four polynomials (P_1, P_2, P_3, P_4) in four unknowns, (x_1, x_2, x_3, x_5) , with (189, 54, 62, 62) terms, respectively. As P_2 has the smallest number of terms, we take the resultant of P_2 with the remaining polynomials while eliminating x_5 to get three polynomials (R_1, R_2, R_3) , where we only keep factors that depend on (x_1, x_2, x_3) . We combine (R_1, R_3) while eliminating x_3 and get a product of polynomials with degrees (8, 8, 48, 96) which depend on (x_1, x_2) . We keep the polynomial of degree 48, which we call V_1 . We also combine (R_2, R_3) while eliminating x_3 and get a product of polynomials with degrees (32, 64) which depend on (x_1, x_2) , and keep the polynomial with degree 32, which we call V_2 . We combine (V_1, V_2) and get a product of polynomials with degrees (2, 4, 4, 4, 4, 16, 32, 56, 536) and of multiplicity (16, 4, 8, 8, 4, 1, 1, 1, 1), respectively. The solutions for x_1 correspond to the zeros of the 16-degree polynomial (with multiplicity 1).

To find x_2 , we substitute each solution of x_1 into V_1 and V_2 and find any common zeros. The remaining joint angles are found in closed form using subproblems. By also including any complex zeros, we find all 16 solutions for q to higher precision than shown in [25].

B.4. General 6R: ABB YuMi with fixed joint 3

For this 7R robot [9], we fix $q_3 = \pi/2$ and choose $R_{06} = I$ and $p_{06} = 50e_x - 355e_y + 354e_z$. Starting with four polynomials with (10, 10, 46, 56) terms, we eliminate three variables. The solutions for x_1 are the eight real zeros of an order 12 polynomial.

Appendix C. Supplementary data

Supplementary material related to this article can be found online at <https://doi.org/10.1016/j.mechmachtheory.2025.105971>.

Data availability

No data was used for the research described in the article.

References

- [1] S. Lu, D. Zlatanov, X. Ding, Approximation of cylindrical surfaces with deployable Bennett networks, *J. Mech. Robot.* 9 (2) (2017) 021001.
- [2] Franka Emika, Franka production 3 product manual, 2022.
- [3] KUKA, LBR iiwa 7 R800, 2023, Datasheet.
- [4] ABB, Product specification - IRB 6640, 2020.
- [5] Universal Robots, User manual UR5/CB3, 2021.
- [6] Kassow Robots, KR series generation 2, 2023.
- [7] FANUC, FANUC robot CRX-10iA, CRX-10iA/L mechanical unit operator's manual, 2020.
- [8] Kawasaki Robotics, Standard specifications KJ125U-B, 2021.
- [9] ABB, Product specification - IRB 14000, 2020.
- [10] Robotics Research Corporation, Dexterous manipulators and advanced control systems, 2005.
- [11] P. Beeson, B. Ames, TRAC-IK: An open-source library for improved solving of generic inverse kinematics, in: 2015 IEEE-RAS 15th International Conference on Humanoid Robots (Humanoids), IEEE, 2015, pp. 928–935.
- [12] R. Diankov, Automated Construction of Robotic Manipulation Programs (Ph.D. thesis), Carnegie Mellon University, 2010.
- [13] A. Malik, Y. Lischuk, T. Henderson, R. Praznica, A deep reinforcement-learning approach for inverse kinematics solution of a high degree of freedom robotic manipulator, *Robotics* 11 (2) (2022) 44.
- [14] B. Ames, J. Morgan, G. Konidakis, IKFlow: Generating diverse inverse kinematics solutions, *IEEE Robot. Autom. Lett.* 7 (3) (2022) 7177–7184.
- [15] D. Ostermeier, J. Külz, M. Althoff, Automatic geometric decomposition for analytical inverse kinematics, 2024, arXiv preprint arXiv:2409.14815.
- [16] D.L. Pieper, The Kinematics of Manipulators Under Computer Control (Ph.D. thesis), Stanford University, 1969.
- [17] R. Murray, Z. Li, S. Sastry, A Mathematical Introduction to Robotic Manipulation, CRC Press, Boca Raton, FL, 1994.
- [18] A.J. Elias, J.T. Wen, Redundancy parameterization and inverse kinematics of 7-DOF revolute manipulators, *Mech. Mach. Theory* 204 (2024) 105824.
- [19] rpiRobotics, IK-Geo, 2024, GitHub repository.
- [20] M. Raghavan, B. Roth, Kinematic analysis of the 6R manipulator of general geometry, in: International Symposium on Robotics Research, Citeseer, 1990, pp. 314–320.
- [21] M. Raghavan, B. Roth, Inverse kinematics of the general 6R manipulator and related linkages, *J. Mech. Des.* 115 (3) (1993) 502–508.
- [22] H. Lee, C. Woernle, M. Hiller, A complete solution for the inverse kinematic problem of the general 6R robot manipulator, *J. Mech. Des.* 113 (4) (1991) 481–486.
- [23] D. Kohli, M. Osvatic, Inverse kinematics of general 6R and 5R, P serial manipulators, *J. Mech. Des.* 114 (4) (1993) 922–931.
- [24] D. Manocha, J.F. Canny, Efficient inverse kinematics for general 6R manipulators, *IEEE Trans. Robot. Autom.* 10 (5) (1994) 648–657.
- [25] M.L. Husty, M. Pfurner, H.-P. Schröcker, A new and efficient algorithm for the inverse kinematics of a general serial 6R manipulator, *Mech. Mach. Theory* 42 (1) (2007) 66–81.
- [26] J. Angeles, Fundamentals of Robotic Mechanical Systems: Theory, Methods, and Algorithms, Springer, New York, 2014.
- [27] T. Rudny, Solving inverse kinematics by fully automated planar curves intersecting, *Mech. Mach. Theory* 74 (2014) 310–318.
- [28] C. Mavroidis, B. Roth, Structural parameters which reduce the number of manipulator configurations, *J. Mech. Des.* 116 (1) (1994) 3–10.
- [29] F. Xiao, G. Li, D. Jiang, Y. Xie, J. Yun, Y. Liu, L. Huang, Z. Fang, An effective and unified method to derive the inverse kinematics formulas of general six-DOF manipulator with simple geometry, *Mech. Mach. Theory* 159 (2021) 104265.
- [30] C. Trinh, D. Zlatanov, M. Zoppi, R. Molfino, A geometrical approach to the inverse kinematics of 6R serial robots with offset wrists, in: International Design Engineering Technical Conferences and Computers and Information in Engineering Conference, vol. 57144, American Society of Mechanical Engineers, 2015, V05CT08A016.
- [31] D. Coleman, I.A. Şucan, S. Chitta, N. Correll, Reducing the barrier to entry of complex robotic software: A MoveIt! case study, *J. Softw. Eng. Robot.* 5 (1) (2014) 3–16.
- [32] ROS-Industrial, Tesseract, 2024.
- [33] D. Zhang, B. Hannaford, IKBT: Solving symbolic inverse kinematics with behavior tree, *J. Artificial Intelligence Res.* 65 (2019) 457–486.
- [34] B.E. Paden, Kinematics and Control of Robot Manipulators (Ph.D. thesis), University of California, Berkeley, 1986.
- [35] W. Kahan, Lectures on computational aspects of geometry, 1983, Unpublished.
- [36] H. Wang, X. Lu, C. Sheng, Z. Zhang, W. Cui, Y. Li, General frame for arbitrary 3R subproblems based on the POE model, *Robot. Auton. Syst.* 105 (2018) 138–145.
- [37] T. Song, B. Pan, G. Niu, J. Yan, Y. Fu, General closed-form inverse kinematics for arbitrary three-joint subproblems based on the product of exponential model, *Front. Mech. Eng.* 17 (2) (2022) 1–17.
- [38] P. Zsombor-Murray, A. Gferrer, 3R wrist positioning – a classical problem and its geometric background, in: Proc. of the 5th Int. Workshop on Computational Kinematics, Springer, Berlin, 2009, pp. 175–182.
- [39] I. Dimovski, M. Trompeska, S. Samak, V. Dukovski, D. Cvetkoska, Algorithmic approach to geometric solution of generalized Paden–Kahan subproblem and its extension, *Int. J. Adv. Robot. Syst.* 15 (1) (2018).
- [40] T. Yue-sheng, X. Ai-Ping, Extension of the second Paden-Kahan sub-problem and its' application in the inverse kinematics of a manipulator, in: 2008 IEEE Conference on Robotics, Automation and Mechatronics, IEEE, 2008, pp. 379–381.
- [41] Q. Chen, S. Zhu, X. Zhang, Improved inverse kinematics algorithm using screw theory for a six-DOF robot manipulator, *Int. J. Adv. Robot. Syst.* 12 (10) (2015) 140.

- [42] H.S. An, T.W. Seo, J.W. Lee, Generalized solution for a sub-problem of inverse kinematics based on product of exponential formula, *J. Mech. Sci. Technol.* 32 (5) (2018) 2299–2307.
- [43] H. Wang, X. Lu, Z. Zhang, Y. Li, C. Sheng, L. Gao, A novel second subproblem for two arbitrary axes of robots, *Int. J. Adv. Robot. Syst.* 15 (2) (2018).
- [44] M. Kong, Z. Du, L. Sun, Y. Zhang, Solution and application of two inverse kinematics subproblems, in: 2006 International Conference on Mechatronics and Automation, IEEE, 2006, pp. 1164–1168.
- [45] J. Xu, Z. Liu, Q. Cheng, Y. Zhao, Y. Pei, C. Yang, Models for three new screw-based IK sub-problems using geometric descriptions and their applications, *Appl. Math. Model.* 67 (2019) 399–412.
- [46] I.-M. Chen, Y. Gao, Closed-form inverse kinematics solver for reconfigurable robots, in: Proceedings 2001 ICRA. IEEE International Conference on Robotics and Automation (Cat. No. 01CH37164), vol. 3, IEEE, 2001, pp. 2395–2400.
- [47] J. Leoro, T. Hsiao, C. Betancourt, A new geometric subproblem to extend solvability of inverse kinematics based on screw theory for 6R robot manipulators, *Int. J. Control. Autom. Syst.* 19 (1) (2021) 562–573.
- [48] J. Zhao, W. Wang, Y. Gao, H. Cai, Generation of closed-form inverse kinematics for reconfigurable robots, *Front. Mech. Eng. China* 3 (1) (2008) 91–96.
- [49] H. Wang, X. Lu, W. Cui, Z. Zhang, Y. Li, C. Sheng, General inverse solution of six-degrees-of-freedom serial robots based on the product of exponentials model, *Assem. Autom.* 38 (3) (2018) 361–367.
- [50] E. Sariyildiz, H. Temeltas, A comparison study of three screw theory based kinematic solution methods for the industrial robot manipulators, in: 2011 IEEE International Conference on Mechatronics and Automation, IEEE, 2011, pp. 52–57.
- [51] Y.S. Tan, P.L. Cheng, A.P. Xiao, Solution for a new sub-problem in screw theory and its' application in the inverse kinematics of a manipulator, in: Applied Mechanics and Materials, vol. 34, Trans Tech Publ, 2010, pp. 271–275.
- [52] Y.S. Tan, P.L. Cheng, A.P. Xiao, Inverse kinematics solution for a 6R special configuration manipulators based on screw theory, in: Advanced Materials Research, vol. 216, Trans Tech Publ, 2011, pp. 250–253.
- [53] J.M. Pardos-Gotor, *Screw Theory in Robotics: An Illustrated and Practicable Introduction to Modern Mechanics*, CRC Press, 2021.
- [54] P.-F. Lin, M.-B. Huang, H.-P. Huang, Analytical solution for inverse kinematics using dual quaternions, *IEEE Access* 7 (2019) 166190–166202.
- [55] L. Chen, T. Zielinska, J. Wang, W. Ge, Solution of an inverse kinematics problem using dual quaternions, *Int. J. Appl. Math. Comput. Sci.* 30 (2) (2020).
- [56] L. Josuet, B. Carlos, L. Hsien, H. Te-Sheng, W. Chun-Sheng, et al., An improved inverse kinematics solution of 6R-DOF robot manipulators with euclidean wrist using dual quaternions, in: 2016 International Automatic Control Conference, CACS, IEEE, 2016, pp. 77–82.
- [57] M.D. Shuster, F.L. Markley, Generalization of the Euler angles, *J. Astronaut. Sci.* 51 (2) (2003) 123–132.
- [58] W.I. Clement, H.H. An, B. Reed, Analytical Shoulder-Elbow-Wrist Inverse Kinematics for the 7-DOF Motoman SIA50D Robot Arm, Tech. Rep., NASA, 2013.
- [59] M. Pfurner, Explicit algebraic solution of geometrically simple serial manipulators, in: Computational Kinematics: Proceedings of the 5th International Workshop on Computational Kinematics, Springer, 2009, pp. 167–174.
- [60] MathWorks, Robotics system toolbox, 2024.
- [61] S. Abbasbandy, S.-j. Liao, A new modification of false position method based on homotopy analysis method, *Appl. Math. Mech.* 29 (2008) 223–228.
- [62] Maplesoft, a division of Waterloo Maple Inc., Maple, 2021.
- [63] Wolfram Research Inc., Mathematica, version 13.3, 2023.
- [64] Maplesoft, a division of Waterloo Maple Inc., How Maple compares to Mathematica, 2015.
- [65] L. Euler, Nova methodus motum corporum rigidorum degerminandi, in: *Novi commentarii academiae scientiarum Petropolitanae*, 1776, pp. 208–238.
- [66] O. Rodrigues, Des lois géométriques qui régissent les déplacements d'un système solide dans l'espace, et de la variation des coordonnées provenant de ces déplacements considérés indépendamment des causes qui peuvent les produire, *J. Math. Appl.* 5 (1840) 380–440.
- [67] P. Zsombor-Murray, M. Hayes, M. Husty, Extreme distance to a spatial circle, *Trans. Can. Soc. Mech. Eng.* 28 (2A) (2004) 221–235.
- [68] A. Norrdine, An algebraic solution to the multilateration problem, in: Proceedings of the 15th International Conference on Indoor Positioning and Indoor Navigation, Sydney, Australia, vol. 1315, 2012.
- [69] B. Odehnal, Common normals of two tori, *J. Geom. Graph.* 9 (1) (2005) 51–65.
- [70] S.L. Shmakov, A universal method of solving quartic equations, *Int. J. Pure Appl. Math.* 71 (2) (2011) 251–259.
- [71] D. Eberly, Intersection of Ellipses, Tech. Rep., Geometric Tools, Redmond WA, 2020.